MINISTERUL EDUCAȚIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

Centrul de Excelenţă în Informatică şi Tehnologii Informaţionale

RAPORT

LA PRACTICA TEHNOLOGICĂ

SPECIALITATEA "Programare şi analiza produselor program"

TEMA : "Hotel management"

A elaborat elevul

Pascalov Alexandr, P-2334r

Conducătorul practicii

Şarapanovscaia Irina

Chişinău 2026

Содержание

Содержание 2
Введение. 4
Постановка задачи 5
Словесное описание 5
Техническое задание по проекту 5
Проектирование базы данных 10
ER диаграмма 10
Реляционная модель 11
Нормализация 12
Создание Базы данных 13
Диаграмма Базы данных 13
Таблицы Базы данных 14
Команды SQL создания БД и таблиц 17
Запросы 21
Представления 23
Хранимые процедуры 25
Создание приложения 30
Авторизация 30
Описание интерфейса приложения 31
Просмотр 32
Редактирование 32
Вычисления 34
Поиск и/или фильтрация 35
Создание отчетов 36
Поддержка приложения 38
Создание резервных копий 38
Восстановление данных 39
Создание установочного файла 40
Руководство пользователя 45

Вывод 48
Общие впечатления от практики 48
Что понравилось 48
Что не понравилось 48
Какие трудности встретил 48
Что помогло 48
Какие есть предложения по практике 48
Библиография 49
Приложение 50

Введение.

Общее описание практики

Технологическая практика преследует достижение общих целей:

- накопление студентами практических навыков и опыта создания общей структуры визуального приложения;

- настройка компонентов визуального приложения;

- управление компонентами визуального приложения;

- разработка визуального приложения в соответствии с техническими условиями;

- поддержка визуальных приложений;

- использование визуального программирования для разработки приложений;

- изучение механизма подключения базы данных к программируемому приложению;

- применение методов восстановления данных и создание резервных копий;

- соблюдение стандартов, касающихся способа оформления документов.

Период практики 05;05;2026 – 01;06;2026

Время практики 11:00 – 13:50

Перечисление использованных инструментов (Visual Studio 2026, MySQL Workbench 8.0[1], draw.io и.д.)

[1] ***MySQL Workbench — официальная графическая среда разработки и администрирования баз данных MySQL от Oracle Corporation. Используется для проектирования схем, выполнения SQL-запросов и управления сервером.***

Постановка задачи

Словесное описание

Эта база данных превращает работу отеля в управляемый и прозрачный процесс. Она автоматически ведёт клиентов, бронирования, номера и услуги, контролирует действия персонала и минимизирует ошибки. Система ускоряет обслуживание гостей, снижает нагрузку на сотрудников и даёт руководству полный набор инструментов для анализа работы отеля. Благодаря автоматизации рутинных операций отель работает быстрее, точнее и эффективнее — как единой, слаженной системой.

Техническое задание по проекту

🎯 Цель задания

Разработать индивидуальный программный проект, включающий проектирование и создание базы данных, разработку приложения с графическим интерфейсом, реализацию операций обработки данных, формирование отчетов, использование представлений и транзакций, создание резервной копии базы данных и подготовку установочного файла приложения.

📌 Общая формулировка задания

В период прохождения практики требуется разработать программное приложение для выбранной предметной области. Приложение должно работать с реляционной базой данных, обеспечивать авторизацию пользователей, выполнение операций добавления, редактирования, удаления и просмотра данных, поиск и фильтрацию, формирование отчетов и дашбордов, а также распределение прав доступа в зависимости от роли пользователя.

Итоговый проект должен быть представлен в виде работающего приложения, базы данных, SQL-сценариев, резервной копии, установочного файла, отчета по практике и исходного кода.

🗄️ 1. Проектирование базы данных

На первом этапе необходимо спроектировать базу данных для выбранной предметной области.

Необходимо выполнить:

- составить словесное описание предметной области;

- определить основные сущности базы данных;
- определить атрибуты каждой сущности;
- указать первичные и внешние ключи;
- определить связи между сущностями;
- разработать ER-диаграмму;
- выполнить нормализацию данных;
- представить реляционную модель базы данных.

Результат: словесное описание предметной области, ER-диаграмма, нормализация, реляционная модель.

🧩 2. Создание базы данных

На основе разработанной модели необходимо создать базу данных и подготовить SQL-сценарии.

Необходимо выполнить:

- создать базу данных;
- создать таблицы согласно реляционной модели;
- задать первичные ключи;
- задать внешние ключи;
- задать ограничения целостности данных;
- создать SQL-сценарии базы данных;
- заполнить таблицы тестовыми данными;
- создать диаграмму базы данных;
- проверить корректность связей между таблицами.

Результат: созданная база данных, SQL-сценарии, диаграмма базы данных, тестовые данные.

⚙️ 3. Создание приложения

Необходимо разработать программное приложение с графическим интерфейсом, подключенное к созданной базе данных.

Приложение должно содержать:

- форму авторизации пользователя;
- главное меню приложения;
- окна или формы для работы с основными таблицами;

- операции CRUD: добавление, просмотр, редактирование и удаление данных;
- поиск данных;
- фильтрацию данных;
- дашборды для отображения обобщенной информации;
- отчеты на основе данных из базы данных;
- распределение прав доступа в зависимости от роли пользователя;
- сообщения об ошибках и успешном выполнении операций;
- проверку корректности вводимых данных.

Результат: рабочее приложение с авторизацией, меню, CRUD-операциями, поиском, фильтрацией, дашбордами, отчетами и ролями пользователей.

👥 4. Авторизация и распределение прав доступа

В приложении необходимо реализовать контроль доступа пользователей к функциональным возможностям системы.

Необходимо предусмотреть:

- таблицу пользователей в базе данных;
- хранение логина, пароля и роли пользователя;
- форму входа в приложение;
- проверку введенных данных пользователя;
- разные права доступа для разных ролей;
- ограничение доступа к отдельным пунктам меню или операциям;
- аудитинг.

Пример ролей: администратор, пользователь, оператор, менеджер.

📊 5. Представления, транзакции и вывод данных

Для удобного вывода и обработки информации необходимо использовать объекты базы данных и механизмы управления данными.

Необходимо реализовать:

- вывод данных через представления;
- создание представлений для отчетов или сложных выборок;
- использование хранимых процедур;
- использование транзакций при выполнении важных операций;

- обработку ошибок при работе с базой данных;
- проверку сохранения целостности данных после выполнения операций.

Результат: представления для вывода данных, SQL-запросы, примеры использования транзакций.

📈 6. Дашборды и отчеты

В приложении необходимо реализовать визуальное представление данных и формирование отчетов.

Необходимо разработать:

- дашборд с основными показателями системы;
- не менее 3 отчетов на основе данных из базы;
- отчет с фильтрацией данных;
- отчет с агрегированными данными: количество, сумма, среднее значение и т.д.;
- возможность просмотра отчета;
- возможность экспорта отчета в доступный формат.

Результат: дашборд, отчеты, экспортированные файлы отчетов.

💾 7. Создание резервной копии базы данных

Необходимо выполнить резервное копирование базы данных для сохранения разработанного проекта и возможности его восстановления.

Необходимо выполнить:

- создать резервную копию базы данных;
- сохранить файл резервной копии в отдельной папке проекта;
- проверить возможность восстановления базы данных;
- описать порядок создания резервной копии в отчете.

Результат: файл резервной копии базы данных и описание процедуры резервного копирования.

📦 8. Создание установочного файла

После завершения разработки необходимо подготовить приложение к установке на другом компьютере.

Необходимо выполнить:

- подготовить финальную версию приложения;
- создать установочный файл приложения;
- проверить запуск приложения после установки;
- убедиться, что приложение корректно подключается к базе данных;
- описать процесс установки в отчете.

Результат: установочный файл приложения и инструкция по установке.

🧪 9. Тестирование проекта

Перед сдачей необходимо протестировать приложение и убедиться в его корректной работе.

Необходимо проверить:

- запуск приложения;
- работу авторизации;
- работу меню;
- добавление данных;
- редактирование данных;
- удаление данных;
- поиск и фильтрацию;
- вывод данных через представления;
- корректность выполнения транзакций;
- формирование отчетов;
- работу дашборда;
- создание и восстановление резервной копии;
- работу приложения после установки.

Результат: протестированная финальная версия программного продукта.

Проектирование базы данных

ER диаграмма

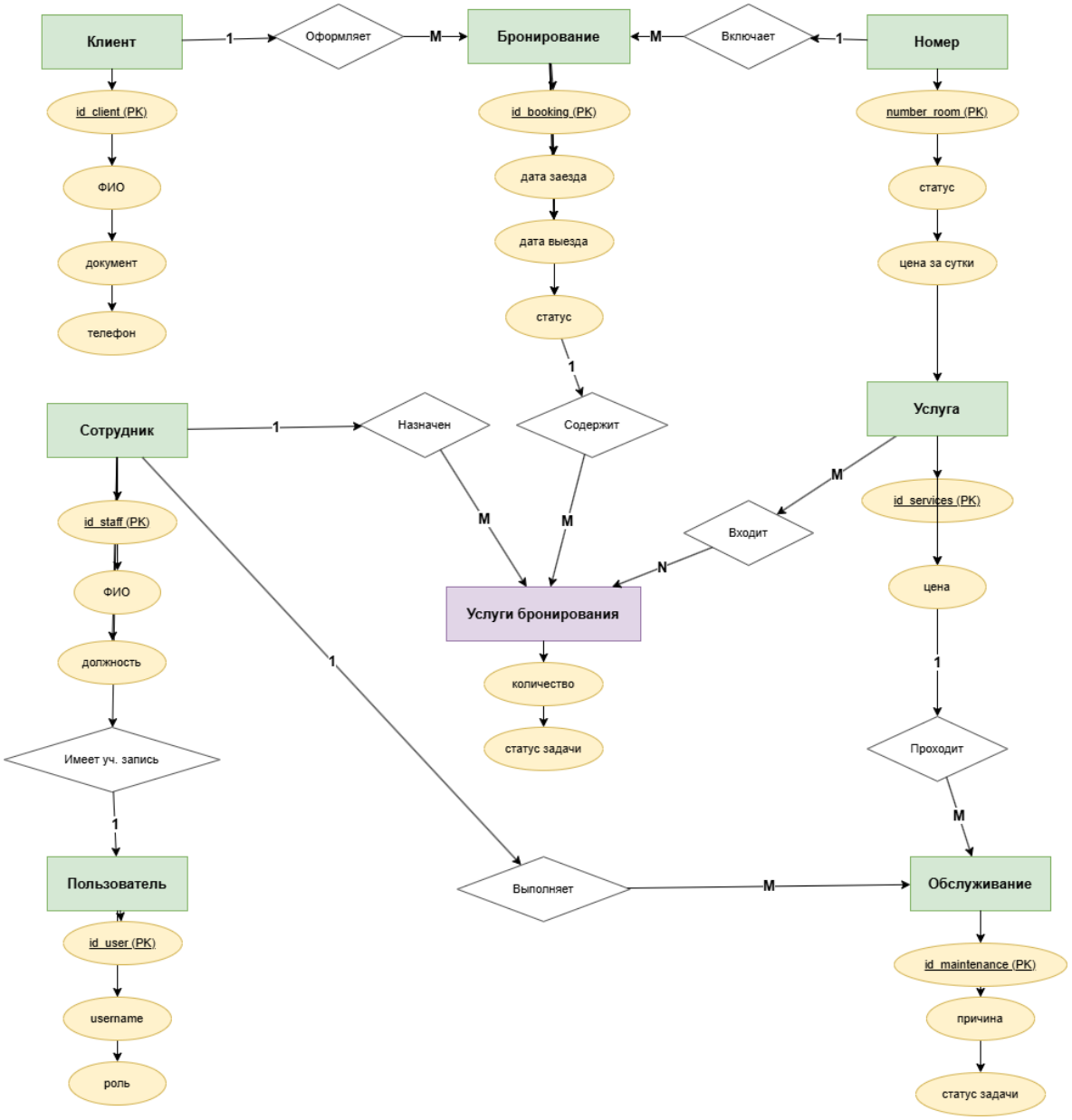


Рисунок 1ER-диаграмма базы данных

Реляционная модель

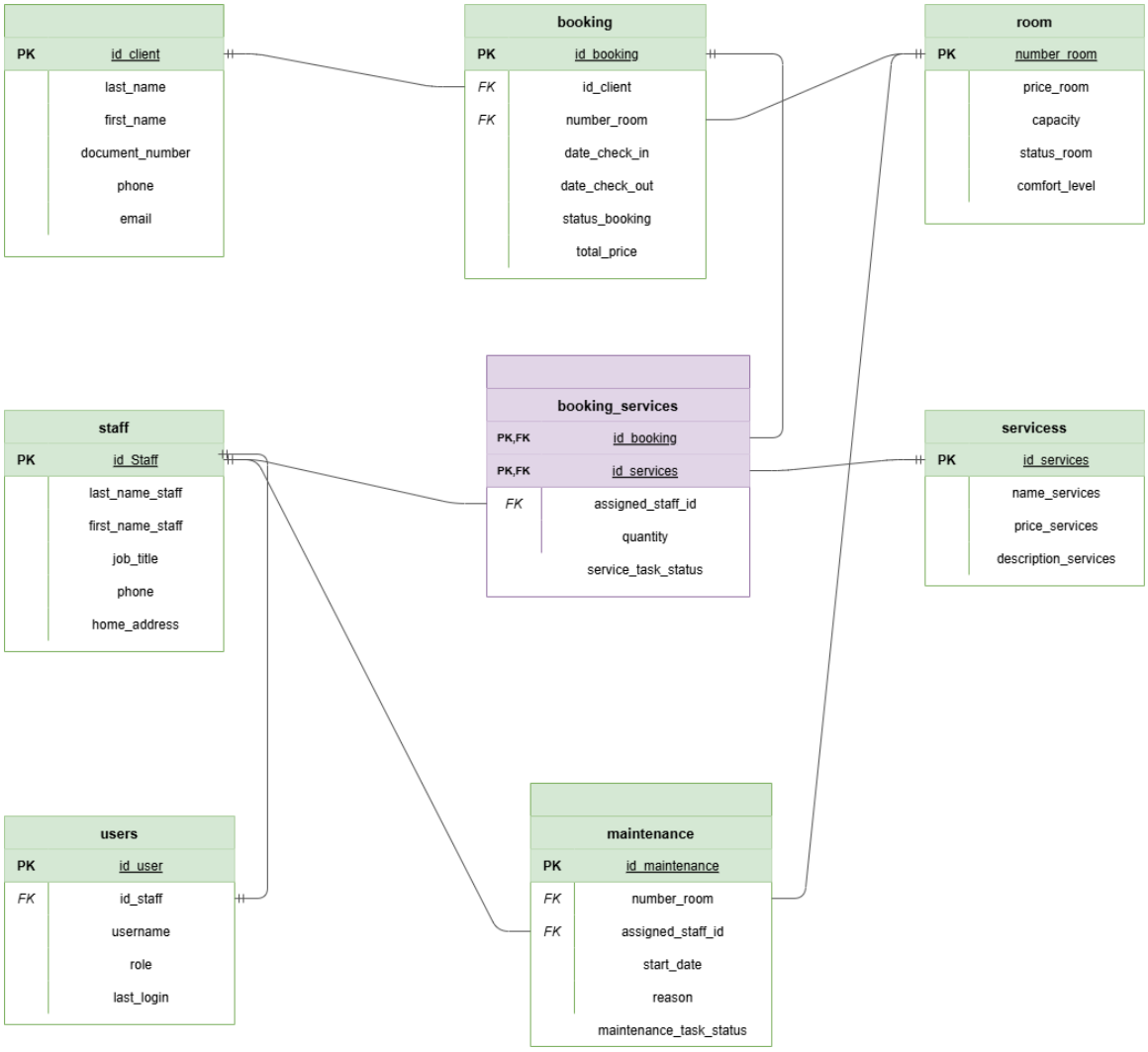


Рисунок 2. Реляционная модель базы данных

Нормализация

Таблица clients приведена к 3НФ. Она хранит только данные клиента. В итоговой модели оставлены основные атрибуты клиента: идентификатор, фамилия, имя, номер документа, телефон и email. В полной физической схеме также есть дополнительные паспортные и служебные поля.

Таблица room приведена к 3НФ. Она хранит только характеристики номера: номер комнаты, цену, вместимость, статус и уровень комфорта.

Таблица booking приведена к 3НФ. Она хранит сам факт бронирования, даты проживания, статус и итоговую стоимость. Данные клиента и номера не дублируются, а подключаются через внешние ключи.

Таблица servicess приведена к 3НФ. Она хранит справочник услуг: название, описание и цену.

Таблица booking_services приведена к 3НФ. Она устраняет связь многие-ко-многим между бронированиями и услугами. Также хранит количество услуги, назначенного сотрудника и статус выполнения задачи.

Таблица staff приведена к 3НФ. Она хранит данные сотрудников: ФИО, должность, телефон и адрес.

Таблица users приведена к 3НФ. Она отделяет учетные записи приложения от данных сотрудников. Пользователь связан с сотрудником через внешний ключ id_staff.

Таблица maintenance приведена к 3НФ. Она хранит задачи обслуживания номеров и связывает их с конкретным номером и ответственным сотрудником.

Создание Базы данных

Диаграмма Базы данных

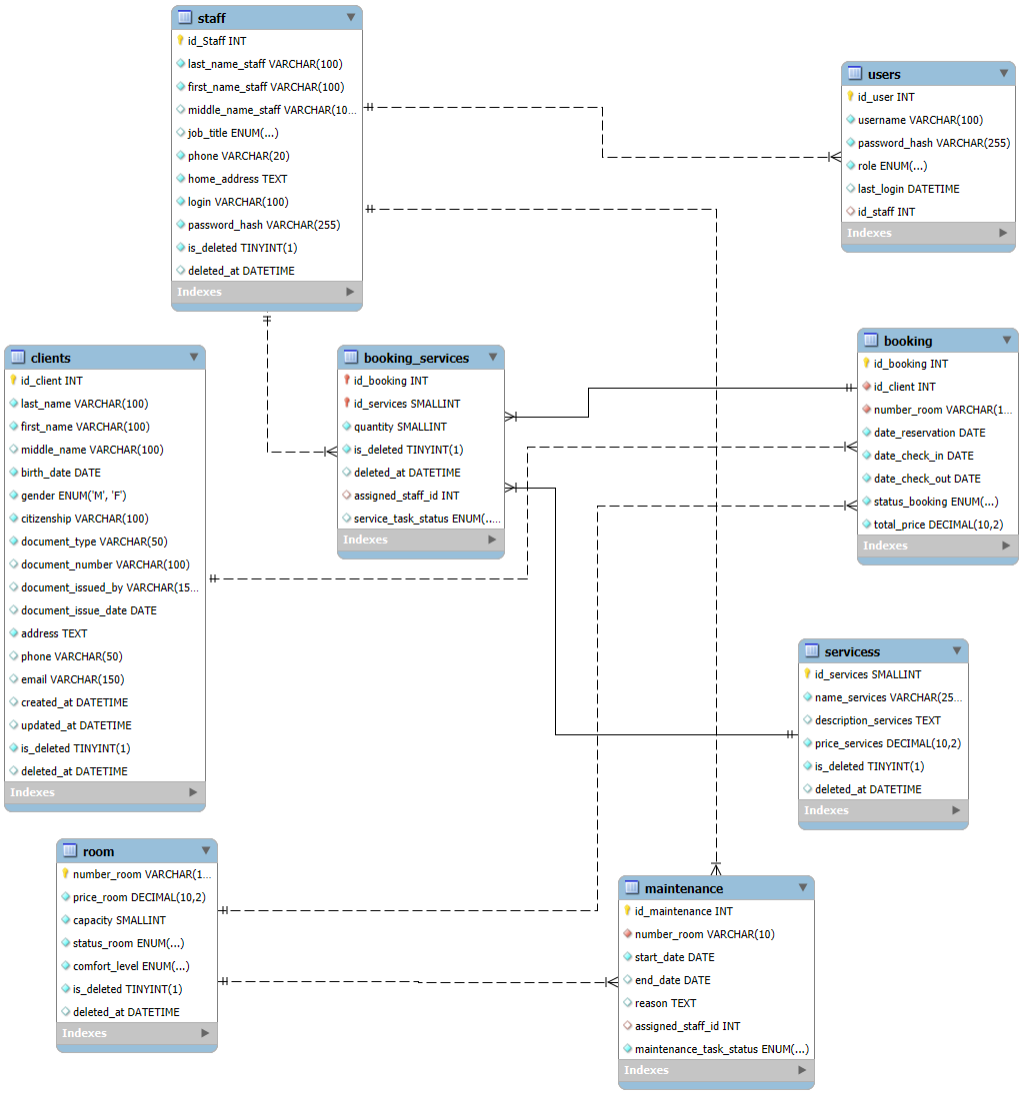


Рисунок 3. Диаграмма Базы данных

Таблицы Базы данных

Table: users

Columns:

id_user	int AI PK
username	varchar(100)
password_hash	varchar(255)
role	enum('admin','manager','receptionist','housekeeper','technician')
last_login	datetime
id_staff	int

Рисунок 4. Таблица Пользователи

Table: staff

Columns:

id_Staff	int AI PK
last_name_staff	varchar(100)
first_name_staff	varchar(100)
middle_name_staff	varchar(100)
job_title	enum('admin','manager','receptionist','housekeeper','technician')
phone	varchar(20)
home_address	text
login	varchar(100)
password_hash	varchar(255)
is_deleted	tinyint(1)
deleted_at	datetime

Рисунок 5. Таблица Работники

Table: servicess

Columns:

id_services	smallint AI PK
name_services	varchar(255)
description_services	text
price_services	decimal(10,2)
is_deleted	tinyint(1)
deleted_at	datetime

Рисунок 6. Таблица Услуги

Table: room

Columns:

number_room	varchar(10) PK
price_room	decimal(10,2)
capacity	smallint
status_room	enum('free','occupied','maintenance')
comfort_level	enum('standard','deluxe','suite')
is_deleted	tinyint(1)
deleted_at	datetime

Рисунок 7. Таблица Комнаты

Table: maintenance

Columns:

id_maintenance	int AI PK
number_room	varchar(10)
start_date	date
end_date	date
reason	text
assigned_staff_id	int
maintenance_task_status	enum('assigned','in_progress','done','problem')

Рисунок 8. Таблица Обслуживания

Table: clients

Columns:

id_client	int AI PK
last_name	varchar(100)
first_name	varchar(100)
middle_name	varchar(100)
birth_date	date
gender	enum('M','F')
citizenship	varchar(100)
document_type	varchar(50)
document_number	varchar(100)
document_issued_by	varchar(150)
document_issue_date	date
address	text
phone	varchar(50)
email	varchar(150)
created_at	datetime
updated_at	datetime
is_deleted	tinyint(1)
deleted_at	datetime

Рисунок 9. Таблица Клиенты

Table: booking_services

Columns:

id_booking	int PK
id_services	smallint PK
quantity	smallint
is_deleted	tinyint(1)
deleted_at	datetime
assigned_staff_id	int
service_task_status	enum('assigned','in_progress','done','problem')

Рисунок 10. Таблица Бронирование услуг

Table: booking

Columns:

id_booking	int AI PK
id_client	int
number_room	varchar(10)
date_reservation	date
date_check_in	date
date_check_out	date
status_booking	enum('reservat','canceled','completed')
total_price	decimal(10,2)

Рисунок 11. Таблица Бронирования

Команды SQL создания БД и таблиц

База данных hotel_bd создаётся с помощью команды CREATE DATABASE с кодировкой utf8mb4[2]. Ниже приведены команды создания базы данных и всех основных таблиц.

```
-- Создание базы данных

CREATE DATABASE IF NOT EXISTS `hotel_bd`
  DEFAULT CHARACTER SET utf8mb4
  COLLATE utf8mb4_0900_ai_ci;

USE `hotel_bd`;

-- Таблица: clients
CREATE TABLE `clients` (
  `id_client` int NOT NULL AUTO_INCREMENT,
  `last_name` varchar(100) NOT NULL,
  `first_name` varchar(100) NOT NULL,
  `middle_name` varchar(100) DEFAULT NULL,
  `birth_date` date NOT NULL,
  `gender` enum('M','F') NOT NULL,
  `citizenship` varchar(100) NOT NULL,
  `document_type` varchar(50) NOT NULL,
  `document_number` varchar(100) DEFAULT NULL,
  `document_issued_by` varchar(150) DEFAULT NULL,
  `document_issue_date` date DEFAULT NULL,
  `address` text NOT NULL,
  `phone` varchar(50) DEFAULT NULL,
  `email` varchar(150) DEFAULT NULL,
  `created_at` datetime DEFAULT CURRENT_TIMESTAMP,
  `updated_at` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  `is_deleted` tinyint(1) NOT NULL DEFAULT '0',
  `deleted_at` datetime DEFAULT NULL,
  PRIMARY KEY (`id_client`),
  UNIQUE KEY `uq_clients_document_number` (`document_number`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

[2] *utf8mb4 — расширение кодировки UTF-8 в MySQL, поддерживающее полный диапазон символов Unicode, включая эмодзи и символы редких языков. Рекомендуется для всех современных баз данных.*

```
-- Таблица: room
CREATE TABLE `room` (
  `number_room` varchar(10) NOT NULL,
  `price_room` decimal(10,2) NOT NULL,
  `capacity` smallint NOT NULL,
  `status_room` enum('free','occupied','maintenance') NOT NULL,
  `comfort_level` enum('standard','deluxe','suite') NOT NULL,
  `is_deleted` tinyint(1) NOT NULL DEFAULT '0',
  `deleted_at` datetime DEFAULT NULL,
  PRIMARY KEY (`number_room`),
  KEY `idx_room_status` (`status_room`),
  KEY `idx_room_comfort` (`comfort_level`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Таблица: booking
CREATE TABLE `booking` (
  `id_booking` int NOT NULL AUTO_INCREMENT,
  `id_client` int NOT NULL,
  `number_room` varchar(10) NOT NULL,
  `date_reservation` date NOT NULL,
  `date_check_in` date NOT NULL,
  `date_check_out` date NOT NULL,
  `status_booking` enum('reservat','canceled','completed') NOT NULL DEFAULT
'reservat',
  `total_price` decimal(10,2) NOT NULL,
  PRIMARY KEY (`id_booking`),
  CONSTRAINT `fk_id_client` FOREIGN KEY (`id_client`) REFERENCES `clients`
(`id_client`),
  CONSTRAINT `fk_number_room` FOREIGN KEY (`number_room`) REFERENCES `room`
(`number_room`),
  CONSTRAINT `chk_booking_dates` CHECK ((`date_check_out` > `date_check_in`))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Таблица: servicess
CREATE TABLE `servicess` (
  `id_services` smallint NOT NULL AUTO_INCREMENT,
  `name_services` varchar(255) NOT NULL,
  `description_services` text,
```

```
  `price_services` decimal(10,2) NOT NULL,
  `is_deleted` tinyint(1) NOT NULL DEFAULT '0',
  `deleted_at` datetime DEFAULT NULL,
  PRIMARY KEY (`id_services`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Таблица: booking_services
CREATE TABLE `booking_services` (
  `id_booking` int NOT NULL,
  `id_services` smallint NOT NULL,
  `quantity` smallint NOT NULL,
  `is_deleted` tinyint(1) NOT NULL DEFAULT '0',
  `deleted_at` datetime DEFAULT NULL,
  `assigned_staff_id` int DEFAULT NULL,
  `service_task_status` enum('assigned','in_progress','done','problem')
DEFAULT 'assigned',
  PRIMARY KEY (`id_booking`,`id_services`),
  CONSTRAINT `fk_id_booking` FOREIGN KEY (`id_booking`) REFERENCES `booking`
(`id_booking`),
  CONSTRAINT `fk_id_services` FOREIGN KEY (`id_services`) REFERENCES
`servicess` (`id_services`),
  CONSTRAINT `booking_services_chk_1` CHECK ((`quantity` > 0))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Таблица: staff
CREATE TABLE `staff` (
  `id_Staff` int NOT NULL AUTO_INCREMENT,
  `last_name_staff` varchar(100) NOT NULL,
  `first_name_staff` varchar(100) NOT NULL,
  `middle_name_staff` varchar(100) DEFAULT NULL,
  `job_title`
enum('admin','manager','receptionist','housekeeper','technician') DEFAULT
'receptionist',
  `phone` varchar(20) NOT NULL,
  `home_address` text NOT NULL,
  `login` varchar(100) NOT NULL,
  `password_hash` varchar(255) NOT NULL,
  `is_deleted` tinyint(1) NOT NULL DEFAULT '0',
  `deleted_at` datetime DEFAULT NULL,
```

```
  PRIMARY KEY (`id_Staff`),
  UNIQUE KEY `login` (`login`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Таблица: users
CREATE TABLE `users` (
  `id_user` int NOT NULL AUTO_INCREMENT,
  `username` varchar(100) NOT NULL,
  `password_hash` varchar(255) NOT NULL,
  `role` enum('admin','manager','receptionist','housekeeper','technician')
NOT NULL,
  `last_login` datetime DEFAULT NULL,
  `id_staff` int DEFAULT NULL,
  PRIMARY KEY (`id_user`),
  UNIQUE KEY `username` (`username`),
  CONSTRAINT `fk_users_staff` FOREIGN KEY (`id_staff`) REFERENCES `staff`
(`id_Staff`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

-- Таблица: maintenance
CREATE TABLE `maintenance` (
  `id_maintenance` int NOT NULL AUTO_INCREMENT,
  `number_room` varchar(10) NOT NULL,
  `start_date` date NOT NULL,
  `end_date` date DEFAULT NULL,
  `reason` text,
  `assigned_staff_id` int DEFAULT NULL,
  `maintenance_task_status` enum('assigned','in_progress','done','problem')
NOT NULL DEFAULT 'assigned',
  PRIMARY KEY (`id_maintenance`),
  CONSTRAINT `fk_number_room_maintenance` FOREIGN KEY (`number_room`)
REFERENCES `room` (`number_room`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Запросы

В системе реализованы SQL-запросы для выборки данных о бронированиях, клиентах и выручке. Запросы используют JOIN, GROUP BY и агрегатные функции (SUM, COUNT, AVG).

```
-- Запрос 1: Получить все активные бронирования с данными клиента и номера
SELECT
    b.id_booking,
    CONCAT(c.last_name, ' ', c.first_name, ' ', COALESCE(c.middle_name,'')) 
AS full_name,
    b.number_room,
    r.comfort_level,
    b.date_check_in,
    b.date_check_out,
    DATEDIFF(b.date_check_out, b.date_check_in) AS nights_count,
    b.total_price,
    b.status_booking
FROM booking b
JOIN clients c ON b.id_client = c.id_client
JOIN room r ON b.number_room = r.number_room
WHERE b.status_booking = 'reservat'
ORDER BY b.date_check_in;

-- Запрос 2: Выручка по месяцам с учётом услуг
SELECT
    DATE_FORMAT(b.date_reservation, '%Y-%m') AS month,
    COUNT(*) AS total_bookings,
    SUM(b.total_price) AS room_revenue,
    SUM(bs.services_total) AS service_revenue,
    SUM(b.total_price + COALESCE(bs.services_total, 0)) AS total_revenue
FROM booking b
LEFT JOIN (
    SELECT id_booking, SUM(quantity * price_services) AS services_total
    FROM booking_services bs
    JOIN servicess s ON bs.id_services = s.id_services
    WHERE bs.is_deleted = 0
    GROUP BY id_booking
) bs ON bs.id_booking = b.id_booking
```

```
        WHERE b.status_booking <> 'canceled'
        GROUP BY DATE_FORMAT(b.date_reservation, '%Y-%m')
        ORDER BY month;

        -- ... (остальные запросы аналогичного характера)

        -- Запрос 3: Статистика загруженности номеров
        SELECT
            r.number_room,
            r.comfort_level,
            COUNT(b.id_booking) AS bookings_count,
            SUM(DATEDIFF(b.date_check_out, b.date_check_in)) AS occupied_nights,
            ROUND(SUM(DATEDIFF(b.date_check_out, b.date_check_in)) / 365 * 100, 2) AS
occupancy_percent
        FROM room r
        LEFT JOIN booking b ON b.number_room = r.number_room
            AND b.status_booking <> 'canceled'
        WHERE r.is_deleted = 0
        GROUP BY r.number_room, r.comfort_level
        ORDER BY occupancy_percent DESC;
```

Представления

Представления (VIEW) скрывают сложность запросов и фильтруют мягко удалённые записи (is_deleted = 0). Приложение работает исключительно через представления, не обращаясь к таблицам напрямую.

```
-- Представление: ActiveClients — все активные (не удалённые) клиенты
CREATE VIEW `activeclients` AS
SELECT
    `clients`.`id_client`,
    `clients`.`last_name`,
    `clients`.`first_name`,
    `clients`.`middle_name`,
    `clients`.`birth_date`,
    `clients`.`gender`,
    `clients`.`citizenship`,
    `clients`.`document_type`,
    `clients`.`document_number`,
    `clients`.`phone`,
    `clients`.`email`,
    `clients`.`is_deleted`,
    `clients`.`deleted_at`
FROM `clients`
WHERE (`clients`.`is_deleted` = 0);

-- Представление: ActiveRooms — все активные номера
CREATE VIEW `activerooms` AS
SELECT
    `room`.`number_room`,
    `room`.`price_room`, (Дэвис, 2015)
    `room`.`capacity`,
    `room`.`status_room`,
    `room`.`comfort_level`,
    `room`.`is_deleted`,
    `room`.`deleted_at`
FROM `room`
WHERE (`room`.`is_deleted` = 0);

-- Представление: vw_Booking — бронирования с полными данными клиента
CREATE VIEW `vw_booking` AS
```

```
        SELECT
            `b`.`id_booking`,
            `b`.`number_room`,
            `b`.`date_reservation`,
            `b`.`date_check_in`,
            `b`.`date_check_out`,
            `b`.`status_booking`,
            `b`.`total_price`,
            CONCAT(`c`.`last_name`,' ',`c`.`first_name`,'
',COALESCE(`c`.`middle_name`,'')) AS `full_name`,
            `c`.`phone`,
            `c`.`email`,
            `c`.`document_type`,
            `c`.`document_number`
        FROM (`booking` `b`
        JOIN `clients` `c` ON (`b`.`id_client` = `c`.`id_client`));

        -- Представление: ActiveBookingService — активные услуги бронирований
        CREATE VIEW `activebookingservice` AS
        SELECT
            `b`.`id_booking`,
            `bs`.`id_services`,
            `b`.`number_room`,
            `b`.`date_check_in`,
            `b`.`date_check_out`,
            `b`.`status_booking`,
            `b`.`total_price` AS `booking_total_price`,
            `s`.`name_services`,
            `s`.`price_services`,
            `bs`.`quantity`,
            (`bs`.`quantity` * `s`.`price_services`) AS `subtotal_service_price`,
            COALESCE(`bs`.`service_task_status`,'assigned') AS `service_task_status`,
            CONCAT(`st`.`last_name_staff`,' ',`st`.`first_name_staff`) AS
`assigned_staff_name`
        FROM `booking_services` `bs`
        JOIN `booking` `b` ON (`bs`.`id_booking` = `b`.`id_booking`)
        JOIN `servicess` `s` ON (`bs`.`id_services` = `s`.`id_services`)
        LEFT JOIN `staff` `st` ON (`bs`.`assigned_staff_id` = `st`.`id_Staff`)
        WHERE (`bs`.`is_deleted` = 0);
```

Хранимые процедуры

Вся бизнес-логика реализована в хранимых процедурах[3]. Они выполняют проверку данных, используют транзакции (START TRANSACTION / COMMIT) для целостности и возвращают понятные сообщения об ошибках.

```
-- Хранимая процедура: AddBooking – создание нового бронирования
CREATE PROCEDURE `AddBooking`(
    IN p_document_number VARCHAR(20),
    IN p_room_number VARCHAR(10),
    IN p_date_check_in DATE,
    IN p_date_check_out DATE
)
BEGIN
    DECLARE v_id_client INT DEFAULT NULL;
    DECLARE v_room_price DECIMAL(10,2) DEFAULT NULL;
    DECLARE v_days INT DEFAULT 0;
    DECLARE v_total_price DECIMAL(10,2) DEFAULT 0;
    DECLARE v_room_exists INT DEFAULT 0;
    DECLARE v_maintenance_conflict INT DEFAULT 0;
    DECLARE v_dates_conflict INT DEFAULT 0;

    SELECT id_client INTO v_id_client
    FROM Clients
    WHERE document_number = p_document_number AND is_deleted = 0 LIMIT 1;

    SELECT COUNT(*), MAX(price_room)
    INTO v_room_exists, v_room_price
    FROM Room WHERE number_room = p_room_number AND is_deleted = 0;

    SELECT COUNT() INTO v_maintenance_conflict
    FROM Maintenance
    WHERE number_room = p_room_number
      AND p_date_check_in <= COALESCE(end_date, DATE '9999-12-31')
      AND p_date_check_out > start_date;

    SELECT COUNT() INTO v_dates_conflict
```

[3] ***Хранимая процедура (Stored Procedure) — заранее скомпилированный и сохранённый на стороне сервера блок SQL-кода, который выполняется по вызову. Обеспечивает инкапсуляцию бизнес-логики и повышает безопасность данных.***

```
    FROM Booking
    WHERE number_room = p_room_number
      AND status_booking = 'reservat'
      AND p_date_check_in < date_check_out
      AND p_date_check_out > date_check_in;

    IF v_id_client IS NULL THEN
        SELECT 'Client not found.' AS ErrorMessage;
    ELSEIF v_room_exists = 0 THEN
        SELECT 'Room not found.' AS ErrorMessage;
    ELSEIF v_maintenance_conflict > 0 THEN
        SELECT 'Room has maintenance for selected dates.' AS ErrorMessage;
    ELSEIF v_dates_conflict > 0 THEN
        SELECT 'Room is already booked for selected dates.' AS ErrorMessage;
    ELSE
        SET v_days = DATEDIFF(p_date_check_out, p_date_check_in);
        SET v_total_price = v_days * v_room_price;

        START TRANSACTION;
        INSERT INTO Booking (id_client, number_room, date_reservation,
            date_check_in, date_check_out, status_booking, total_price)
        VALUES (v_id_client, p_room_number, CURRENT_DATE,
            p_date_check_in, p_date_check_out, 'reservat', v_total_price);
        COMMIT;

        SELECT 'Booking successfully created.' AS Message,
               v_total_price AS TotalPrice, v_days AS NightsCount;
    END IF;
END;

-- Хранимая процедура: AddClient – добавление нового клиента
CREATE PROCEDURE `AddClient`(
    IN p_last_name VARCHAR(100),
    IN p_first_name VARCHAR(100),
    IN p_document_number VARCHAR(50),
    IN p_phone VARCHAR(20),
    IN p_email VARCHAR(100)
)
BEGIN
```

```
        DECLARE v_doc_active INT DEFAULT 0;
        DECLARE v_doc_deleted INT DEFAULT 0;

        SELECT COUNT() INTO v_doc_active
        FROM Clients WHERE document_number = p_document_number AND is_deleted =
0;

        SELECT COUNT() INTO v_doc_deleted
        FROM Clients WHERE document_number = p_document_number AND is_deleted =
1;

        IF v_doc_active > 0 THEN
            SELECT 'Client with this document number already exists.' AS
ErrorMessage;
        ELSEIF v_doc_deleted > 0 THEN
            -- Восстанавливаем удалённого клиента
            START TRANSACTION;
            UPDATE Clients SET last_name = p_last_name, first_name =
p_first_name,
                phone = p_phone, email = p_email, is_deleted = 0, deleted_at =
NULL
            WHERE document_number = p_document_number AND is_deleted = 1;
            COMMIT;
            SELECT 'Client successfully restored.' AS Message;
        ELSE
            START TRANSACTION;
            INSERT INTO Clients (last_name, first_name, document_number, phone,
email)
            VALUES (p_last_name, p_first_name, p_document_number, p_phone,
p_email);
            COMMIT;
            SELECT 'Client successfully added.' AS Message;
        END IF;
    END;

    -- Хранимая процедура: UpdateStaff — обновление данных сотрудника
    CREATE PROCEDURE `UpdateStaff`(
        IN p_id_staff INT,
        IN p_last_name_staff VARCHAR(100),
```

```
            IN p_first_name_staff VARCHAR(100),
            IN p_job_title
ENUM('admin','receptionist','manager','housekeeper','technician'),
            IN p_phone VARCHAR(20),
            IN p_login VARCHAR(100)
        )
        BEGIN
            DECLARE v_staff_exists INT DEFAULT 0;
            DECLARE v_phone_exists INT DEFAULT 0;
            DECLARE v_current_role VARCHAR(20) DEFAULT NULL;
            DECLARE v_admin_count INT DEFAULT 0;

            SELECT COUNT() INTO v_staff_exists
            FROM Staff WHERE id_Staff = p_id_staff AND is_deleted = 0;

            SELECT COUNT() INTO v_phone_exists
            FROM Staff WHERE phone = p_phone AND id_Staff != p_id_staff AND
is_deleted = 0;

            SELECT job_title INTO v_current_role FROM Staff WHERE id_Staff =
p_id_staff;

            SELECT COUNT() INTO v_admin_count
            FROM Staff WHERE job_title = 'admin' AND is_deleted = 0;

            IF v_staff_exists = 0 THEN
                SELECT 'Staff member not found.' AS ErrorMessage;
            ELSEIF v_phone_exists > 0 THEN
                SELECT 'Staff member with this phone already exists.' AS
ErrorMessage;
            ELSEIF v_current_role = 'admin' AND p_job_title != 'admin' AND
v_admin_count <= 1 THEN
                SELECT 'Cannot change role: this is the last admin.' AS ErrorMessage;
            ELSE
                START TRANSACTION;
                UPDATE Staff SET
                    last_name_staff = p_last_name_staff,
                    first_name_staff = p_first_name_staff,
                    job_title = p_job_title,
```

```
                phone = p_phone,
                login = p_login
            WHERE id_Staff = p_id_staff;
            COMMIT;
            SELECT 'Staff member successfully updated.' AS Message;
        END IF;
    END;

    -- Хранимая процедура: DeleteClient — мягкое удаление клиента
    CREATE PROCEDURE `DeleteClient`(
        IN p_id_client INT
    )
    BEGIN
        DECLARE v_client_exists INT DEFAULT 0;
        DECLARE v_active_bookings INT DEFAULT 0;

        SELECT COUNT() INTO v_client_exists
        FROM Clients WHERE id_client = p_id_client AND is_deleted = 0;

        SELECT COUNT() INTO v_active_bookings
        FROM Booking WHERE id_client = p_id_client AND status_booking =
'reservat';

        IF v_client_exists = 0 THEN
            SELECT 'Client not found.' AS ErrorMessage;
        ELSEIF v_active_bookings > 0 THEN
            SELECT 'Cannot delete client with active bookings.' AS ErrorMessage;
        ELSE
            START TRANSACTION;
            UPDATE Clients SET is_deleted = 1, deleted_at = NOW()
            WHERE id_client = p_id_client;
            COMMIT;
            SELECT 'Client successfully deleted.' AS Message;
        END IF;
    END;
```

Создание приложения

Авторизация

При запуске приложения пользователю отображается форма авторизации. Форма содержит два режима: вход в систему и регистрация нового пользователя. Для входа необходимо ввести логин и пароль. Система проверяет введённые данные по базе данных: если логин и хэш пароля совпадают с записью в таблице users, пользователь получает доступ к главному окну приложения с правами, соответствующими его роли (admin, receptionist, manager, housekeeper, technician). При неверных данных выводится сообщение об ошибке. Для регистрации требуется ввести логин, пароль и подтверждение пароля — система валидирует все поля и создаёт нового пользователя с ролью manager. Пароли хранятся в базе данных в виде хэша PBKDF2-SHA256 с 120 000 итерациями.

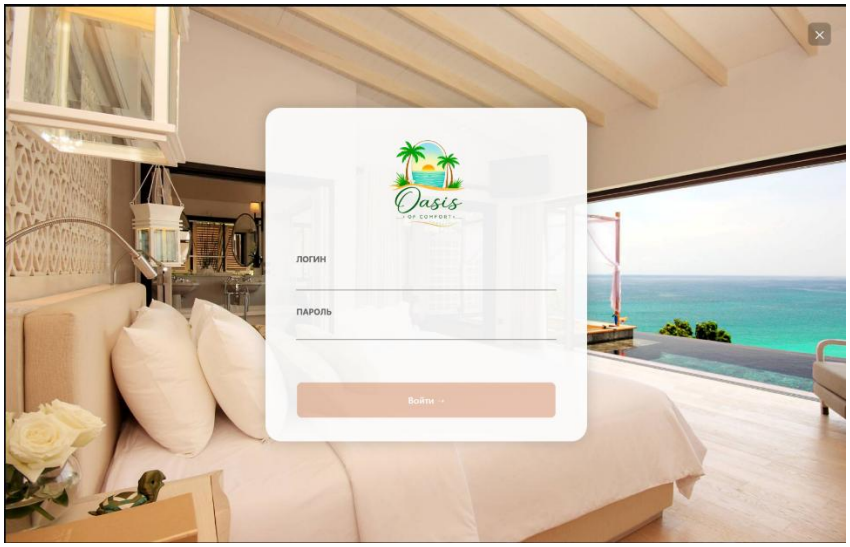


Рисунок 12. Форма авторизации

Фрагмент кода (AuthViewModel.cs — метод аутентификации):

```
private async Task LoginAsync()
{
    _loginAttempted = true;
    ClearLoginMessages();

    if (!ValidateLoginFields(showErrors: true))
    {
        return;
    }

    IsBusy = true;

    try
    {
        var user = await _userService.AuthenticateAsync(LoginUsername.Trim(),
LoginPassword);
        if (user == null)
        {
            LoginError = "Неверный логин или пароль.";
            return;
```

```
            }

            AuthenticatedUser = user;
            AuthenticationCompleted?.Invoke();
        }
        catch (Exception ex)
        {
            LoginError = ex.Message;
        }
        finally
        {
            IsBusy = false;
        }
    }
```

Описание интерфейса приложения

Главное окно приложения построено на основе вкладочной панели с 9 разделами: Клиенты, Номера, Сотрудники, Услуги, Бронирования, Обслуживание, Услуги бронирований, Пользователи и Отчёты. Доступность разделов зависит от роли пользователя. В верхней части окна отображается имя текущего пользователя и его роль. Доступны кнопки: Дашборд, Справка по ролям, Резервная копия, Выход.

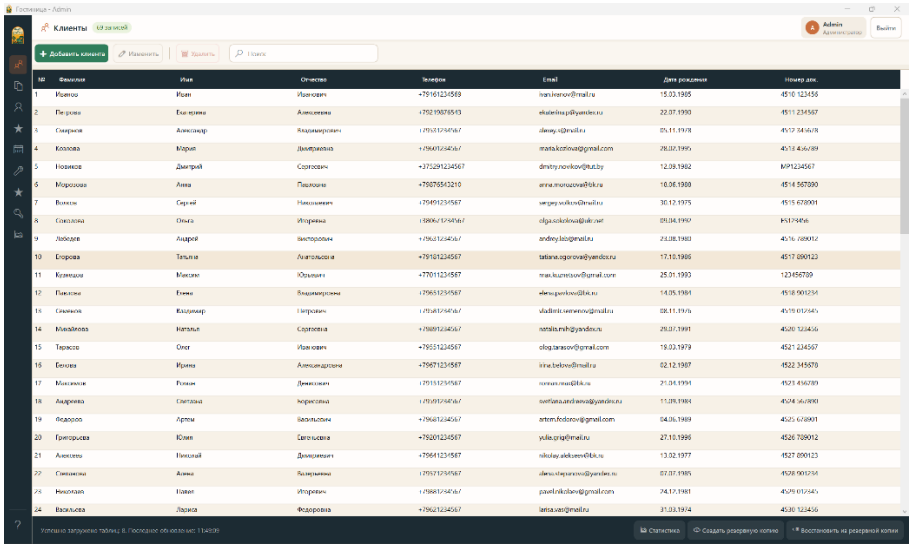


Рисунок 13. Главное окно приложения

Фрагмент кода (MainViewModel.cs — навигация по вкладкам):

```
public string CurrentPageTitle => SelectedMainTabIndex switch
{
    0 => "Клиенты",
    1 => "Номера",
    2 => "Сотрудники",
    3 => "Услуги",
    4 => "Бронирования",
    5 => "Обслуживание",
    6 => "Услуги бронирований",
    7 => "Пользователи",
    8 => "Отчёты",
    _ => "Панель"
};
```

Просмотр

Каждый раздел приложения отображает данные в виде таблицы (DataGrid). При загрузке вкладки автоматически вызывается метод LoadAsync(), который запрашивает данные из базы через сервисный слой и заполняет коллекцию ObservableCollection. Раздел «Клиенты» выводит список всех активных клиентов с полями: фамилия, имя, номер документа, телефон, email. Раздел «Номера» отображает номер комнаты, цену, вместимость, уровень комфорта и текущий статус.

№	Фамилия	Имя	Отчество	Телефон	Email	Дата рождения	Номер док.
1	Иванов	Иван	Иванович	+79161234569	ivan.ivanov@mail.ru	15.03.1985	4510 123456
2	Петрова	Екатерина	Алексеевна	+79219876543	ekaterina.p@yandex.ru	22.07.1990	4511 234567
3	Смирнов	Александр	Владимирович	+79531234567	alexey.s@mail.ru	05.11.1978	4512 345678
4	Козлова	Мария	Дмитриевна	+79601234567	maria.kozlova@gmail.com	28.02.1995	4513 456789
5	Новиков	Дмитрий	Сергеевич	+375291234567	dmitry.novikov@tut.by	12.09.1982	MP1234567
6	Морозова	Анна	Павловна	+79876543210	anna.morozova@bk.ru	18.06.1988	4514 567890
7	Волков	Сергей	Николаевич	+79491234567	sergey.volkov@mail.ru	30.12.1975	4515 678901
8	Соколова	Ольга	Игоревна	+380671234567	olga.sokolova@ukr.net	09.04.1992	FS123456
9	Лебедев	Андрей	Викторович	+79631234567	andrey.lebl@mail.ru	23.08.1980	4516 789012
10	Егорова	Татьяна	Анатольевна	+79181234567	tatiana.egorova@yandex.ru	17.10.1986	4517 890123
11	Кузнецов	Максим	Юрьевич	+77011234567	max.kuznetsov@gmail.com	25.01.1993	123456789
12	Павлова	Елена	Владимировна	+79651234567	elena.pavlova@bk.ru	14.05.1984	4518 901234
13	Семенов	Владимир	Петрович	+79581234567	vladimir.semenov@mail.ru	08.11.1976	4519 012345
14	Михайлова	Наталья	Сергеевна	+79891234567	natalia.mih@yandex.ru	29.07.1991	4520 123456
15	Тарасов	Олег	Иванович	+79551234567	oleg.tarasov@gmail.com	19.03.1979	4521 234567
16	Белова	Ирина	Александровна	+79671234567	irina.belova@mail.ru	02.12.1987	4522 345678
17	Максимов	Роман	Денисович	+79151234567	roman.max@bk.ru	21.04.1994	4523 456789
18	Андреева	Светлана	Борисовна	+79591234567	svetlana.andreeva@yandex.ru	11.09.1983	4524 567890
19	Федоров	Артем	Васильевич	+79681234567	artem.fedorov@gmail.com	04.06.1989	4525 678901
20	Григорьева	Юлия	Геннадьевна	+79201234567	yulia.grig@mail.ru	27.10.1996	4526 789012
21	Алексеев	Николай	Дмитриевич	+79641234567	nikolay.alekseev@bk.ru	13.02.1977	4527 890123
22	Степанова	Алена	Валерьевна	+79571234567	alena.stepanova@yandex.ru	07.07.1985	4528 901234
23	Николаев	Павел	Игоревич	+79881234567	pavel.nikolaev@gmail.com	24.12.1981	4529 012345
24	Васильева	Лариса	Федоровна	+79621234567	larisa.vas@mail.ru	31.03.1974	4530 123456

Рисунок 14. Отображение списка клиентов

Фрагмент кода (ClientsViewModel.cs — загрузка данных):

Редактирование

Операции добавления, редактирования и удаления доступны через кнопки на панели инструментов. Добавление и редактирование открывают отдельное всплывающее окно с формой ввода. Все поля проходят валидацию перед сохранением. Удаление реализовано как мягкое (soft delete): запись не удаляется физически, а получает признак is_deleted = 1. Перед удалением система проверяет наличие связанных активных записей (например, нельзя удалить клиента с активными бронированиями).

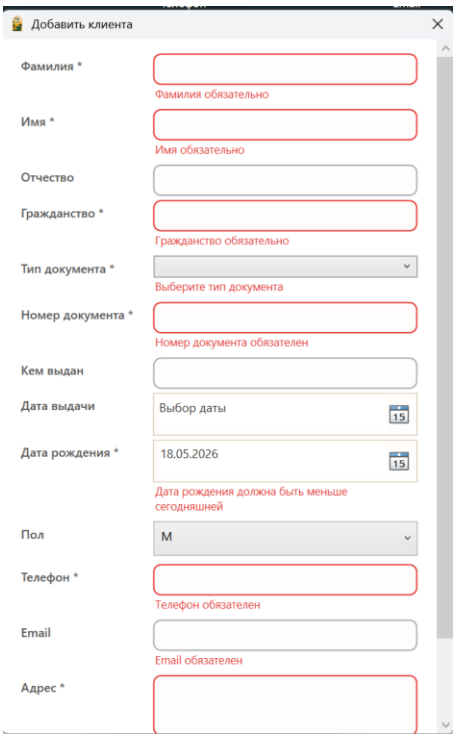


Рисунок 15. Форма добавления клиента

Фрагмент кода (ClientsViewModel.cs — удаление клиента):

```
private async Task RemoveClientAsync()
{
    if (SelectedClient == null)
    {
        MessageBox.Show("Выберите клиента для удаления.", "Информация",
MessageBoxButton.OK, MessageBoxImage.Information);
        return;
    }

    if (SelectedClient.Id <= 0)
    {
        MessageBox.Show("Некорректный ID клиента.", "Ошибка", MessageBoxButton.OK,
MessageBoxImage.Error);
        return;
    }

    var displayEmail = string.IsNullOrWhiteSpace(SelectedClient.Email) ? "—" :
SelectedClient.Email;
    var result = MessageBox.Show(
        $"Вы уверены что хотите удалить клиента?\n\n{SelectedClient.LastName}
{SelectedClient.FirstName}\nEmail: {displayEmail}",
        "Подтверждение",
        MessageBoxButton.YesNo,
        MessageBoxImage.Question);

    if (result != MessageBoxResult.Yes)
    {
        return;
    }

    var (success, message) = await _clientService.DeleteClientAsync(SelectedClient.Id);
    if (!success)
    {
        MessageBox.Show(message, "Ошибка", MessageBoxButton.OK, MessageBoxImage.Error);
        return;
    }
await RefreshAllAsync();}
```

Вычисления

Дашборд предоставляет сводную статистику по работе отеля. Отображается общий доход за выбранный период, средняя цена номера, показатели загрузки номерного фонда (свободные, занятые, на обслуживании), а также количество активных бронирований. Данные вычисляются на стороне базы данных через хранимые процедуры и выводятся на панели дашборда.

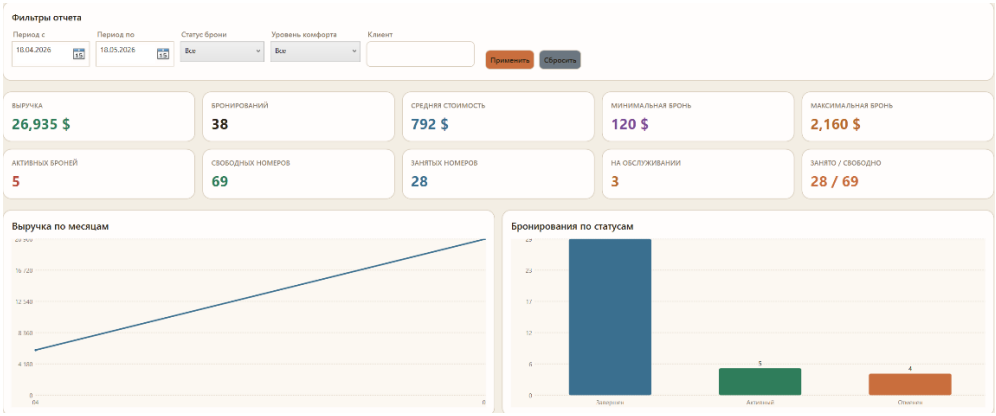


Рисунок 16. Дашборд приложения

Фрагмент кода (DashboardViewModel.cs — поля статистики):

```
private decimal _totalRevenue;
 private decimal _avgPrice;
 private decimal _minPrice;
 private decimal _maxPrice;
 private int _totalBookings;
 private int _activeBookings;
 private int _freeRooms;
 private int _occupiedRooms;
 private int _maintenanceRooms;
```

Поиск и/или фильтрация

В разделах «Клиенты» и «Бронирования» реализован поиск с задержкой 300 мс (debounce) через DispatcherTimer — запрос в базу отправляется только после остановки ввода. Раздел «Номера» поддерживает одновременную фильтрацию по трём параметрам: вместимость, уровень комфорта и статус. Раздел «Бронирования» позволяет фильтровать по статусу и диапазону дат.

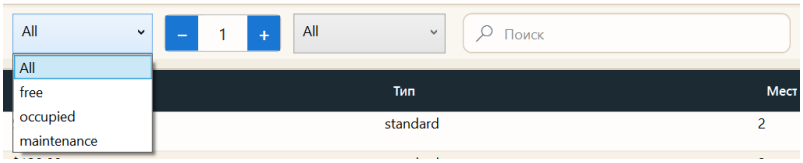


Рисунок 17. Фильтрация номеров

Фрагмент кода (RoomsViewModel.cs — фильтрация номеров):

```
        private async Task ApplyFilterAsync()
        {
            _searchTimer.Stop();

            try
            {
                if (RoomCapacityFilter <= 0)
                {
                    return;
                }

                var results = await _roomService.FilterRooms(
                    RoomCapacityFilter,
                    RoomComfortLevelFilter,
                    RoomStatusFilter,
                    RoomSearchText);

                UpdateCollection(Rooms, results);
            }
            catch (Exception ex)
            {
                SetStatus($"Ошибка при фильтрации номеров: {ex.Message}");
            }
        }
    }
}
```

Создание отчетов

Раздел «Отчёты» позволяет сформировать шесть видов отчётов: Финансовый отчет, Отчет по бронированиям, Загрузка номеров, Клиенты, Услуги, Обслуживание номеров. Для каждого отчёта задаются период (дата начала — дата окончания) и выбирается статус бронирования для фильтрации. Отчёт выводит агрегированные данные: общую сумму дохода, количество бронирований и средний чек. Результаты отображаются в таблице и могут быть экспортированы в CSV или распечатаны.

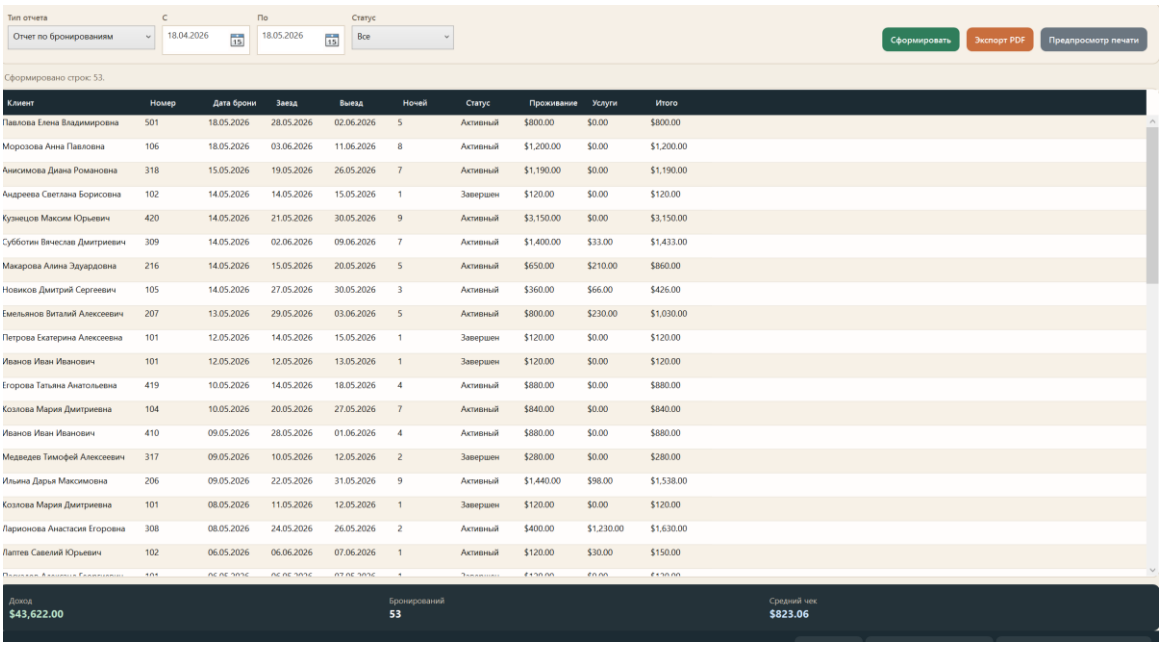


Рисунок 18. Раздел «Отчёты»

Фрагмент кода (ReportsViewModel.cs — типы отчётов и фильтры):

```
public ObservableCollection<string> ReportTypes { get; } = new()
{
    "Финансовый отчет",
    "Отчет по бронированиям",
    "Загрузка номеров",
    "Клиенты",
    "Услуги",
    "Обслуживание номеров"
};
```

Таблица 1 Финансовый отчет

Доход	Бронирований	Средний чек
$43,622.00	53	$823.06

Таблица 2 Загрузка номеров

Доход	Бронирований	Доход / бронь
$26,935.00	34	$792.21

Таблица 3 Услуги

Доход от услуг	Использований	Средний чек услуги
$3,055.00	22	$138.86

Таблица 4 Обслуживание номеров

Записей	Дней всего	В среднем дней
5,00	10	2,00

Поддержка приложения

Создание резервных копий

С помощью соответствующей кнопки на панели управления пользователь может создавать резервные копии базы данных. Программа выдает диалоговое окно, в котором можно будет выбрать путь сохранения и имя файла бэкапа базы данных.

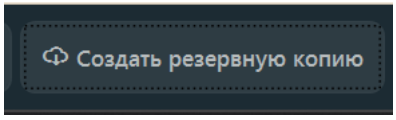


Рисунок 19. Кнопка "Создать резервную копию"

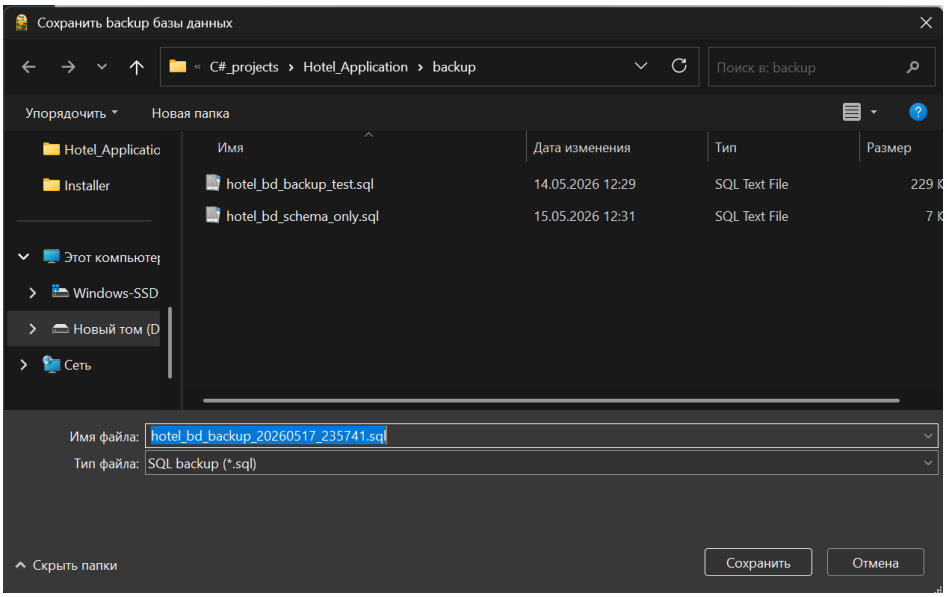


Рисунок 20. Раздел для выбора сохранения копий

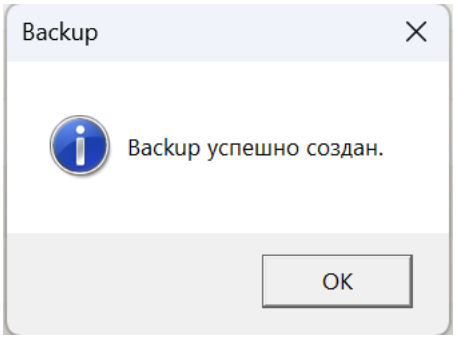


Рисунок 21. Message Box

Восстановление данных

Восстановление происходит аналогично созданию резервной копии. Пользователь выбирает в диалоговом окне файл резервной копии, и программа восстанавливает базу данных.

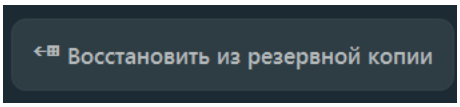


Рисунок 22. Кнопка “Восстановить из резервной копии”

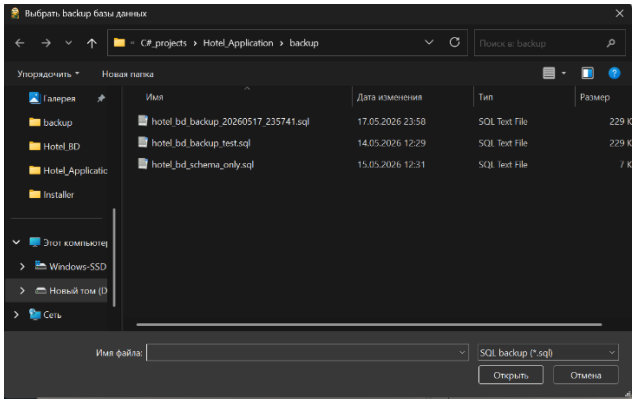


Рисунок 23. Раздел для выбора копий

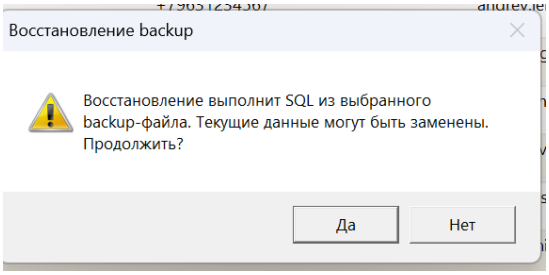


Рисунок 24. Message Box

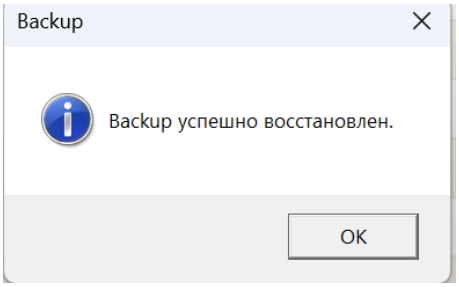


Рисунок 25. Message Box

Создание установочного файла

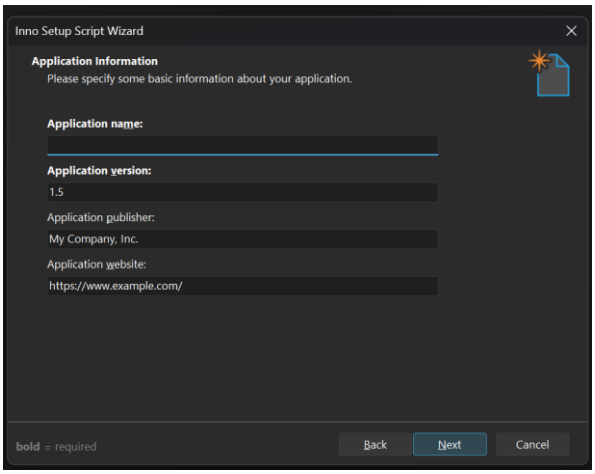


Рисунок 26. Шаг 1

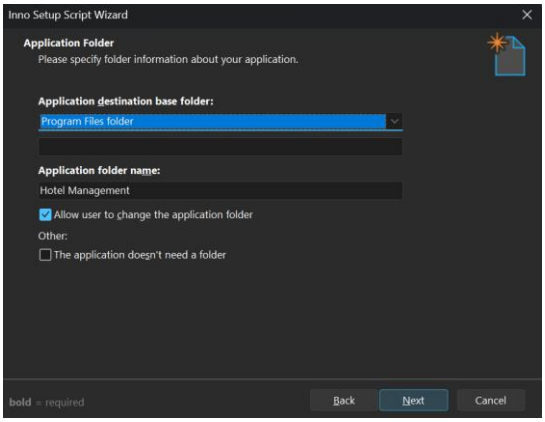


Рисунок 27. Шаг 2

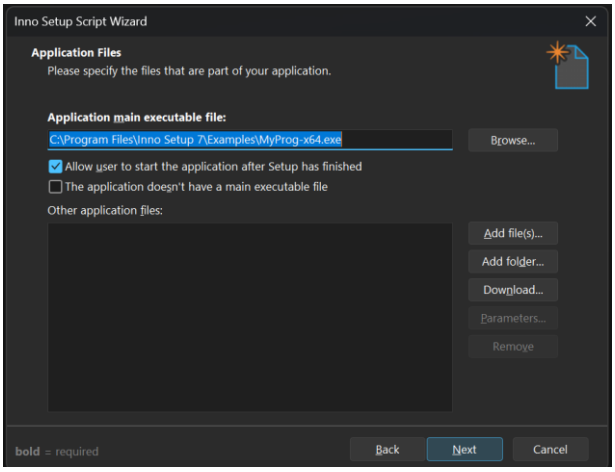


Рисунок 28. Шаг 3

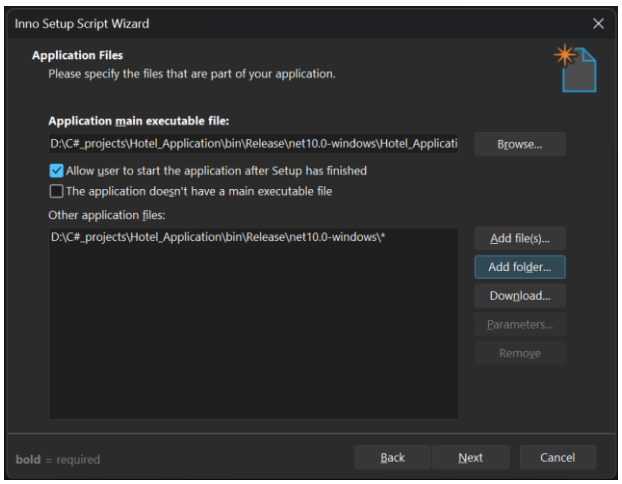


Рисунок 29. Шаг 3

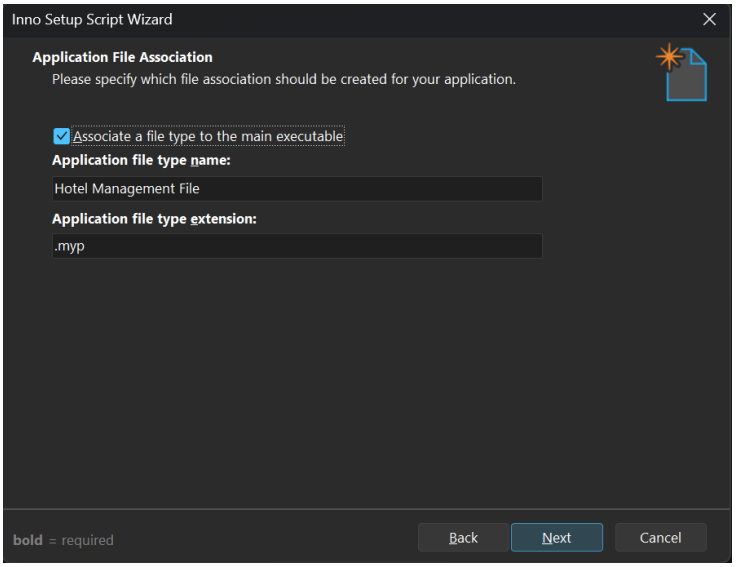


Рисунок 30. Шаг 4

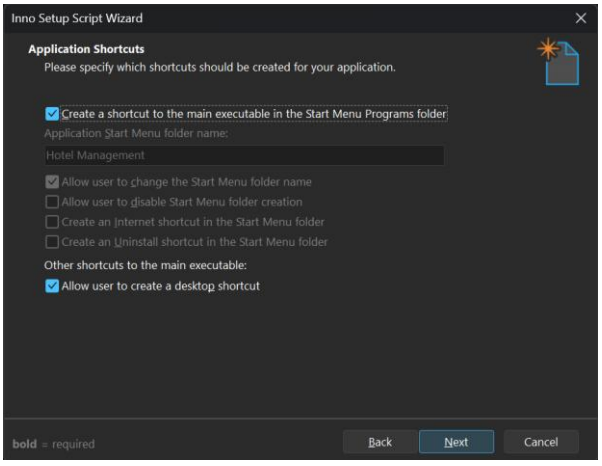


Рисунок 31. Шаг 5

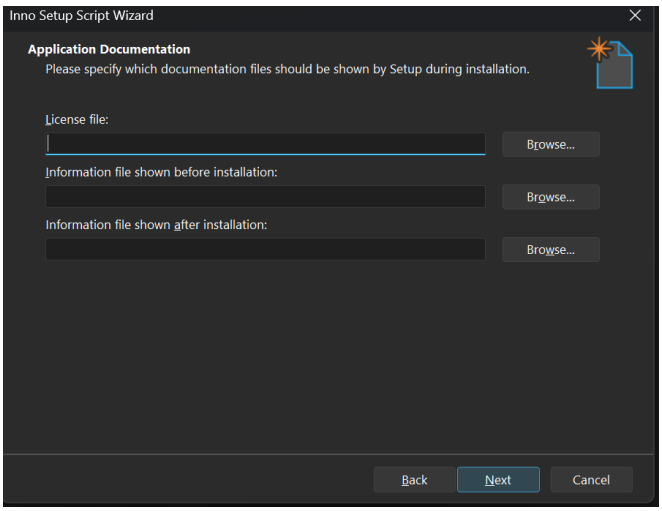


Рисунок 32. Шаг 6

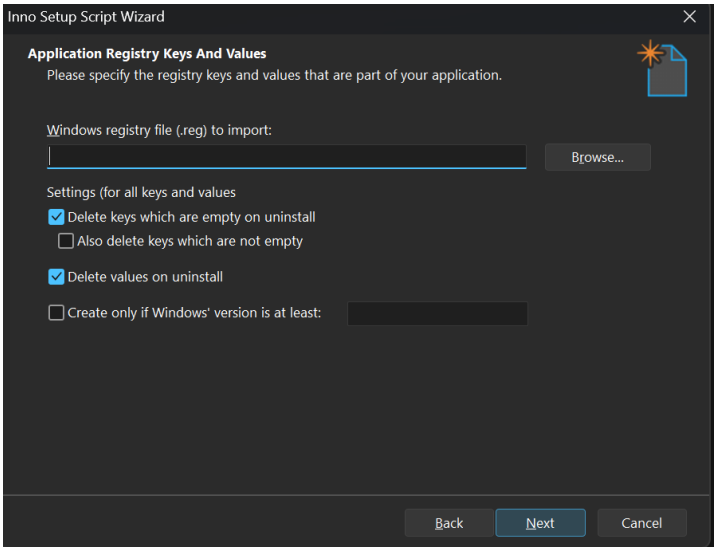


Рисунок 33. Шаг 7

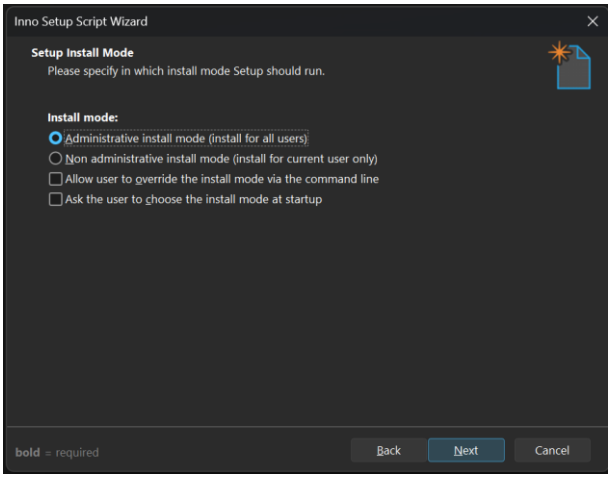


Рисунок 34. Шаг 8

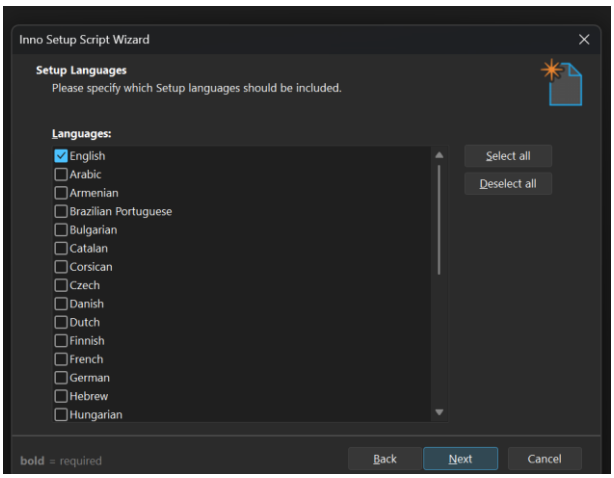


Рисунок 35. Шаг 9

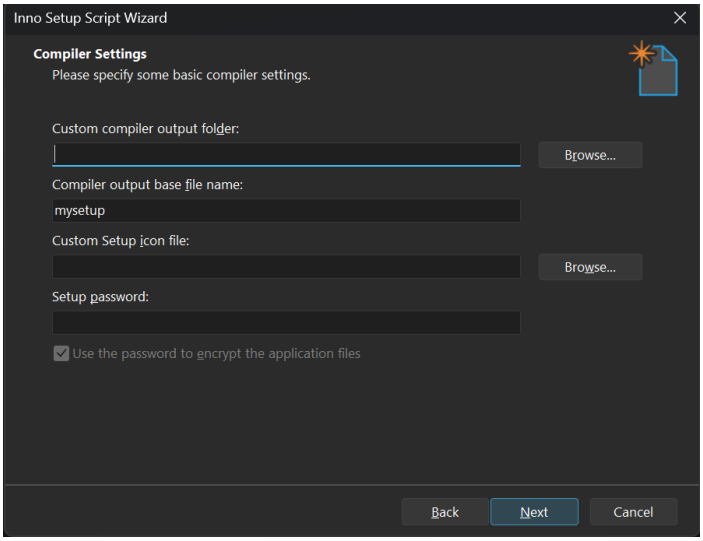


Рисунок 36. Шаг 10

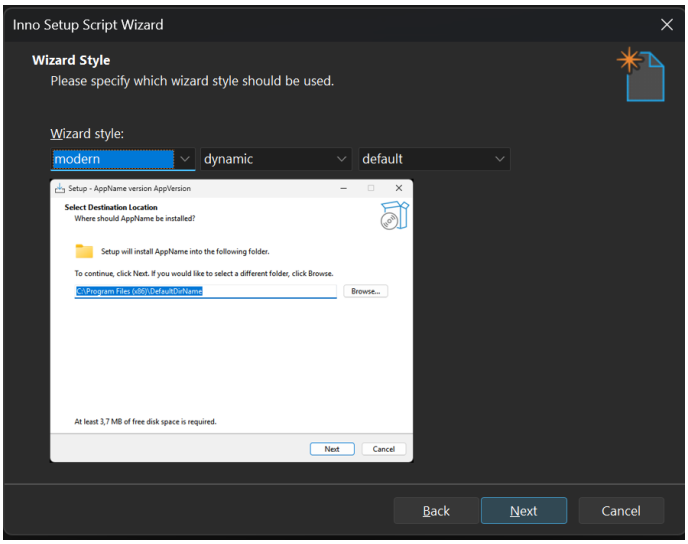


Рисунок 37. Шаг 11

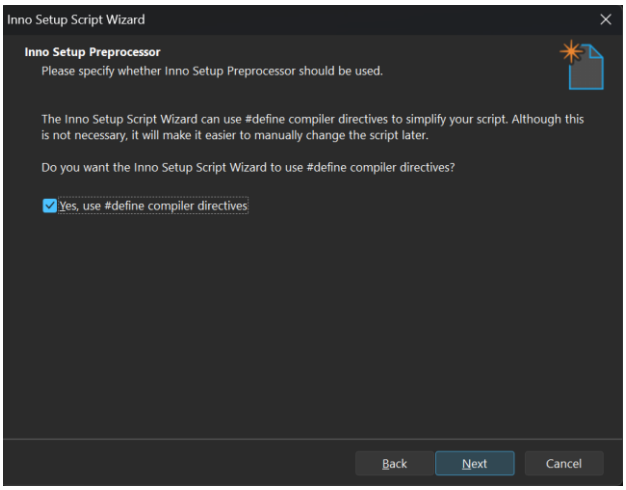


Рисунок 38. Шаг 12

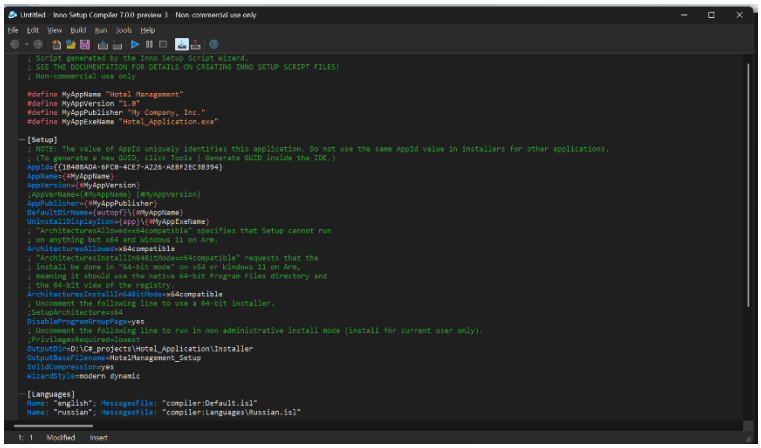


Рисунок 39. Шаг 13

Руководство пользователя

Описание пунктов меню или Help

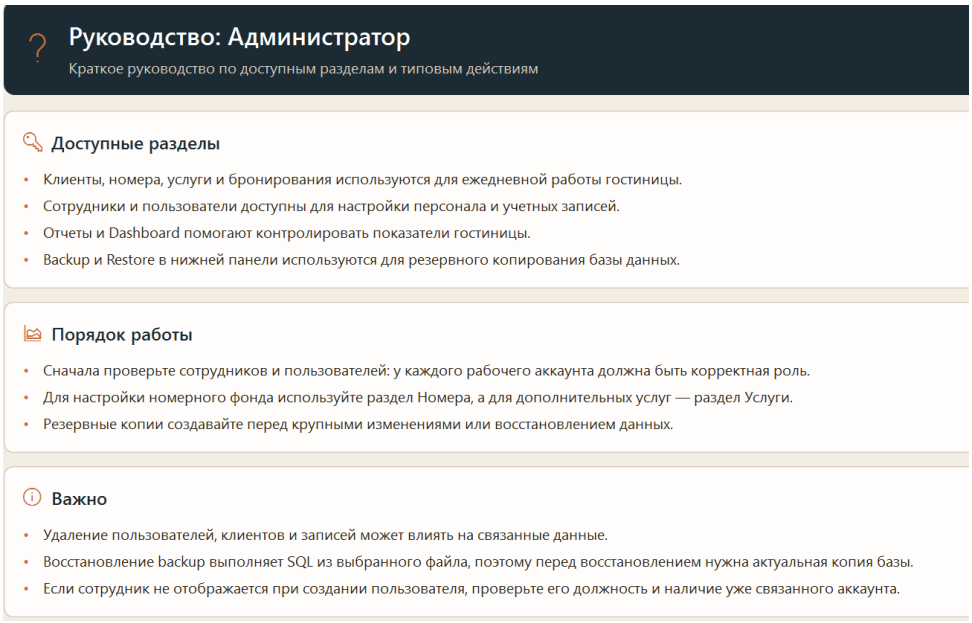


Рисунок 39. Руководство для администратора

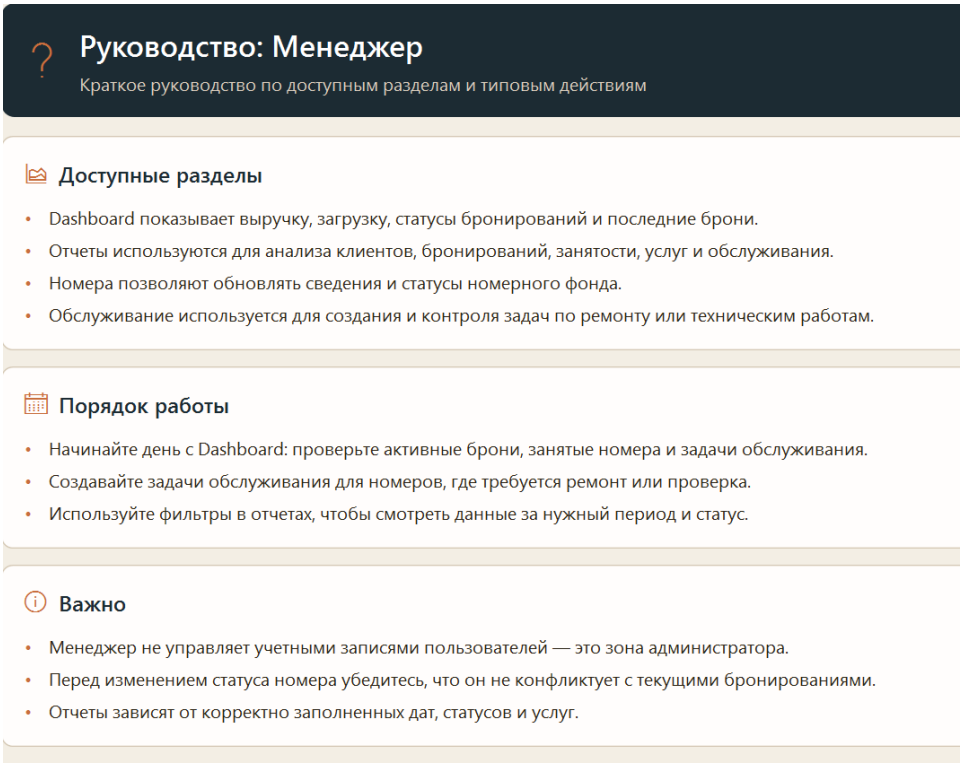


Рисунок 40. Руководство для менеджера

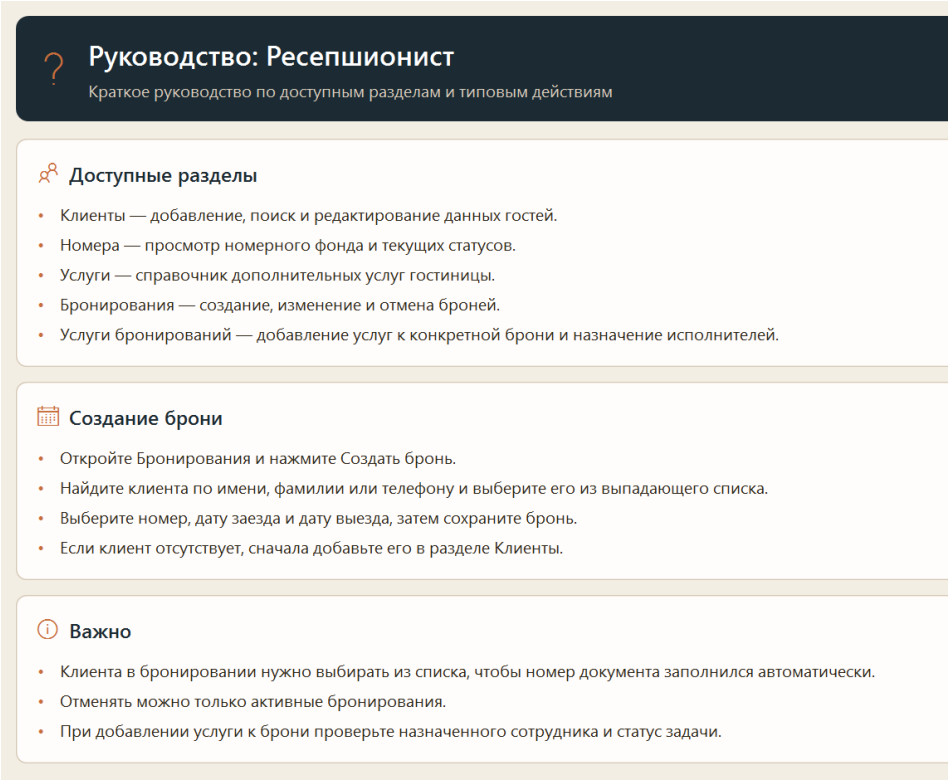


Рисунок 41. Руководство для ресептиониста

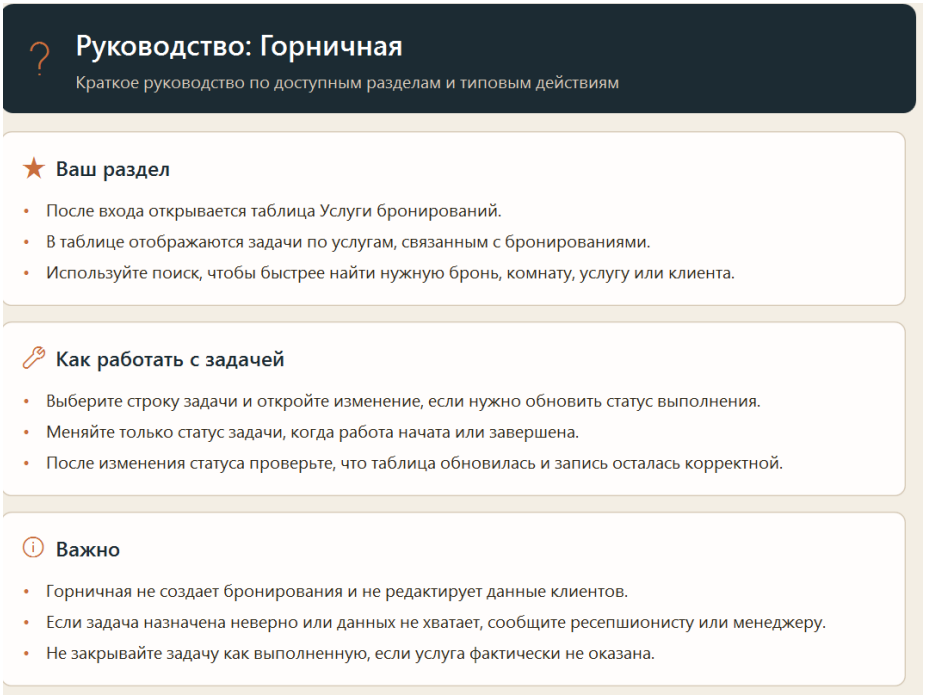


Рисунок 42. Руководство для горничной

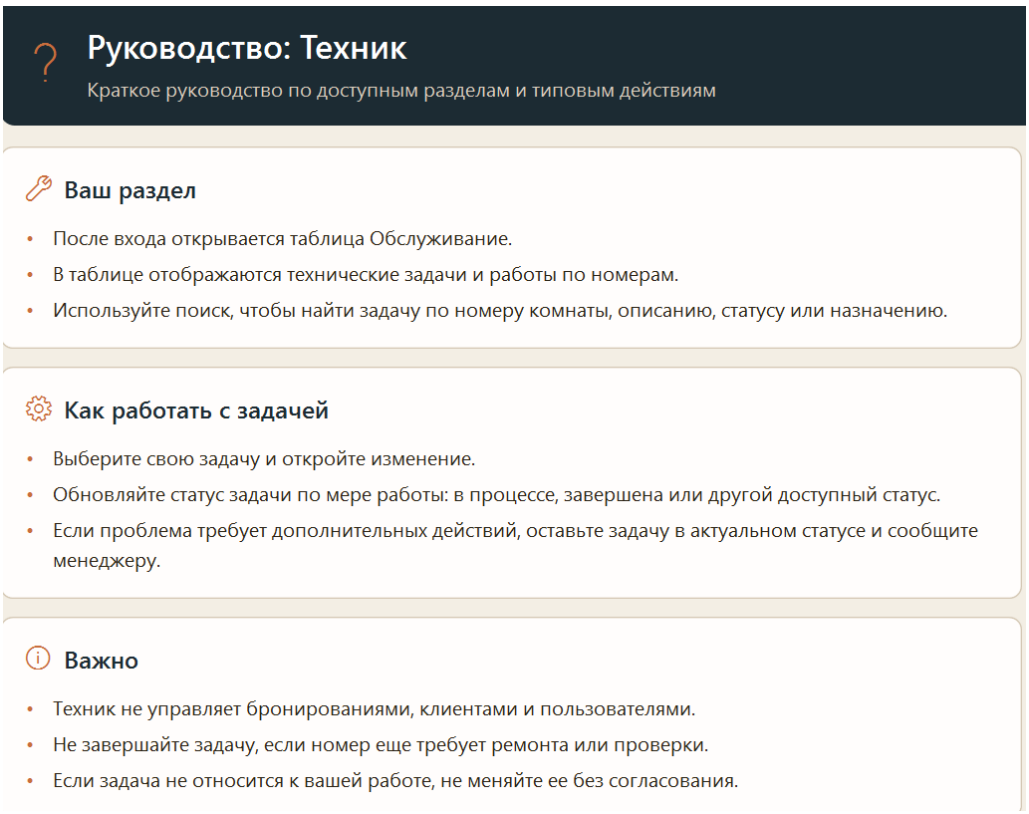


Рисунок 43. Руководство для техника

Вывод

Общие впечатления от практики

Практика подарила бесценный опыт создания собственного продукта, который поможет мне в становлении в качестве профессионала.

Что понравилось

Понравилась свобода творчества в создании проекта.

Что не понравилось

Отсутствие необходимого оборудования, и условий для работы.

Какие трудности встретил

Отсутствие необходимого оборудования, и условий для работы. Т.к я работал на своем ноутбуке мне приходилось носить с собой удлинитель так как в аудиторий, где я находился со своими коллегами не хватало розеток на всех. И приходилось работать через свой интернет т.к интернет колледжа был очень слабеньким и работать чрез него было невозможно.

Что помогло

Использовали несколько общих удлинителей и просили интернет у коллег у которых было много его

Какие есть предложения по практике

Добавить возможность студентам работать в группе над одним проектам для получения навыков работы в команде.

Библиография

1. Кузнецов С. Д. Базы данных. — М.: Академия, 2019. — 496 с.

2. Троелсен Э., Джепикс Ф. Язык программирования C# 9 и платформа .NET 5. — СПб.: Питер, 2022. — 1120 с.

3. Microsoft SQL Server Documentation. [Электронный ресурс]. — URL:

4. C# Guide / Microsoft Docs. [Электронный ресурс]. — URL:

5. Windows Forms .NET Documentation. [Электронный ресурс]. — URL:

6. Microsoft.Data.SqlClient — NuGet Package. [Электронный ресурс]. — URL:

Приложение

Код программы

```
using System.Windows;
using System.Windows.Controls;
using Hotel_Application.Models;
using Hotel_Application.Services;
using Hotel_Application.ViewModels;

namespace Hotel_Application
{
    public partial class MainWindow : Window
    {
        private MainViewModel? _viewModel;

        public MainWindow()
        {
            InitializeComponent();
        }

        public MainWindow(DatabaseManager databaseManager, User currentUser)
            : this()
        {
            _viewModel = new MainViewModel(databaseManager, currentUser);
            DataContext = _viewModel;
            _viewModel.Status = "Подключение успешно установлено.";
            Title = $"Гостиница - {currentUser.Username}";
        }

        private void DataGrid_LoadingRow(object sender, DataGridRowEventArgs e)
        {
            e.Row.Header = (e.Row.GetIndex() + 1).ToString();
        }

        private void ListBox_SelectionChanged(object sender, SelectionChangedEventArgs e)
        {

        }
    }
}
using System;
using System.Collections.Generic;
using System.Collections.Specialized;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Input;
using Hotel_Application.Helpers;
using Hotel_Application.Models;
using Hotel_Application.Repositories;
using Hotel_Application.Services;
using Hotel_Application.Views;
using Microsoft.Win32;
namespace Hotel_Application.ViewModels
{
    public class MainViewModel : ViewModelBase
    {
        private readonly DatabaseManager _databaseManager;
        private readonly DatabaseBackupService _backupService;
        private readonly string _role;
        private bool _isLoading;
        private string _status = "Готово";
        private string _currentUsername = string.Empty;
        private string _currentUserRole = string.Empty;
        private int _selectedMainTabIndex;

        public ClientsViewModel ClientsTab { get; }
```

```
public RoomsViewModel RoomsTab { get; }
public EmployeesViewModel EmployeesTab { get; }
public ServicesViewModel ServicesTab { get; }
public BookingsViewModel BookingsTab { get; }
public UsersViewModel UsersTab { get; }
public MaintenanceViewModel MaintenanceTab { get; }
public BookingServicesViewModel BookingServicesTab { get; }
public ReportsViewModel ReportsTab { get; }

public ICommand LoadDataCommand { get; }
public ICommand OpenDashboardCommand { get; }
public ICommand OpenRoleHelpCommand { get; }
public ICommand CreateBackupCommand { get; }
public ICommand RestoreBackupCommand { get; }
public ICommand LogoutCommand { get; }

public bool IsLoading
{
    get => _isLoading;
    private set
    {
        _isLoading = value;
        OnPropertyChanged();
        CommandManager.InvalidateRequerySuggested();
    }
}

public string Status
{
    get => _status;
    set { _status = value; OnPropertyChanged(); }
}

public string CurrentUsername
{
    get => _currentUsername;
    private set { _currentUsername = value; OnPropertyChanged(); }
}

public string CurrentUserRole
{
    get => _currentUserRole;
    private set { _currentUserRole = value; OnPropertyChanged(); }
}

public string CurrentUser => CurrentUsername;
public string CurrentRole => CurrentUserRole;
public string UserInitials => string.IsNullOrWhiteSpace(CurrentUsername)
    ? "U"
    : CurrentUsername.Trim()[0].ToString().ToUpperInvariant();

public string CurrentPageTitle => SelectedMainTabIndex switch
{
    0 => "Клиенты",
    1 => "Номера",
    2 => "Сотрудники",
    3 => "Услуги",
    4 => "Бронирования",
    5 => "Обслуживание",
    6 => "Услуги бронирований",
    7 => "Пользователи",
    8 => "Отчёты",
    _ => "Панель"
};

public string CurrentPageIcon => SelectedMainTabIndex switch
{
    0 => "\uE716",
    1 => "\uE8C8",
```

```
            2 => "\uE77B",
            3 => "\uE735",
            4 => "\uE787",
            5 => "\uE90F",
            6 => "\uE735",
            7 => "\uE8D7",
            8 => "\uE9D2",
            _ => "\uE80F"
        };

        public int CurrentCount => SelectedMainTabIndex switch
        {
            0 => ClientsTab.Clients.Count,
            1 => RoomsTab.Rooms.Count,
            2 => EmployeesTab.Employees.Count,
            3 => ServicesTab.HotelServices.Count,
            4 => BookingsTab.Bookings.Count,
            5 => MaintenanceTab.Maintenances.Count,
            6 => BookingServicesTab.BookingLinks.Count,
            7 => UsersTab.Users.Count,
            _ => 0
        };

        public int SelectedMainTabIndex
        {
            get => _selectedMainTabIndex;
            set
            {
                _selectedMainTabIndex = value;
                OnPropertyChanged();
                OnCurrentPageMetadataChanged();
            }
        }
    public bool TabClients => SessionService.IsFrontDesk;
     public bool TabRooms => SessionService.IsFrontDesk;
     public bool TabBookings => SessionService.IsFrontDesk;
     public bool TabBookingServices => SessionService.IsFrontDesk
         || SessionService.IsHousekeeper;
     public bool TabServices => SessionService.IsFrontDesk;
     public bool TabMaintenance => SessionService.IsManager || SessionService.IsTechnician;
     public bool TabEmployees => SessionService.IsAdmin;
     public bool TabUsers => SessionService.IsAdmin;
     public bool TabReports => SessionService.IsManager;
     public bool TabDashboard => SessionService.IsManager;

     public bool CanEdit => SessionService.IsFrontDesk;
     public bool CanEditAdminOnly => SessionService.IsAdmin;

     public MainViewModel(DatabaseManager databaseManager, User currentUser)
     {
         _databaseManager = databaseManager;
         _backupService = new DatabaseBackupService(_databaseManager);
         CurrentUsername = currentUser.Username;
         _role = currentUser.Role;
         CurrentUserRole = FormatRole(_role);

         var clientService = new ClientService(_databaseManager);
         var roomService = new RoomService(new RoomRepository(_databaseManager));
         var employeeService = new EmployeeService(_databaseManager);
         var hotelServiceService = new HotelServiceService(_databaseManager);
         var bookingService = new BookingService(_databaseManager);
         var userService = new UserService(_databaseManager);
         var maintenanceService = new MaintenanceService(_databaseManager);
         var bookingServiceLinkService = new BookingServiceLinkService(_databaseManager);
         var reportService = new ReportService(_databaseManager);
         var exportService = new ExportService();
         var reportPrintService = new ReportPrintService();

         ClientsTab = new ClientsViewModel(clientService, SetStatus, LoadData);
```

```
        RoomsTab = new RoomsViewModel(roomService, SetStatus, LoadData);
        EmployeesTab = new EmployeesViewModel(employeeService, SetStatus, LoadData);
        ServicesTab = new ServicesViewModel(hotelServiceService, SetStatus, LoadData);
        BookingsTab = new BookingsViewModel(bookingService, roomService, clientService,
SetStatus, LoadData);
        UsersTab = new UsersViewModel(userService, SetStatus, LoadData);
        MaintenanceTab = new MaintenanceViewModel(maintenanceService, roomService,
employeeService, SetStatus, LoadData);
        ReportsTab = new ReportsViewModel(reportService, exportService, reportPrintService);
        BookingServicesTab = new BookingServicesViewModel(
            bookingServiceLinkService,
            bookingService,
            hotelServiceService,
            employeeService,
            SetStatus,
            LoadData);
        ObserveCurrentCountCollections();
        SelectedMainTabIndex = GetDefaultTabIndex(_role);

        LoadDataCommand = new AsyncRelayCommand(_ => LoadData());
        OpenDashboardCommand = new RelayCommand(_ => OpenDashboard());
        OpenRoleHelpCommand = new RelayCommand(_ => OpenRoleHelp());
        CreateBackupCommand = new AsyncRelayCommand(_ => CreateBackupAsync(), _ =>
CanEditAdminOnly && !IsLoading);
        RestoreBackupCommand = new AsyncRelayCommand(_ => RestoreBackupAsync(), _ =>
CanEditAdminOnly && !IsLoading);
        LogoutCommand = new RelayCommand(_ => Logout());

        _ = LoadData();
    }

    private void Logout()
    {
        SessionService.Clear();

        var userService = new UserService(_databaseManager);
        var authViewModel = new AuthViewModel(userService);

        // Важна правильная подписка или передача DatabaseManager для authViewModel
        // В данном случае мы можем передать DatabaseManager как параметр для AuthWindow,
        // если настроим AuthWindow чтобы оно помнило его для MainWindow,
        // либо можно просто передать action

        var loginWindow = new AuthWindow(authViewModel, _databaseManager);
        loginWindow.Show();

        var currentWindow = Application.Current.MainWindow;
        Application.Current.MainWindow = loginWindow;
        currentWindow?.Close();
    }

    private async Task LoadData()
    {
        if (IsLoading)
        {
            return;
        }

        IsLoading = true;
        Status = "Загрузка всех данных из БД...";

        try
        {
            var loadTasks = new List<Task>();

            if (TabClients) loadTasks.Add(ClientsTab.LoadAsync());
            if (TabRooms) loadTasks.Add(RoomsTab.LoadAsync());
            if (TabEmployees) loadTasks.Add(EmployeesTab.LoadAsync());
            if (TabServices) loadTasks.Add(ServicesTab.LoadAsync());
```

```
            if (TabBookings) loadTasks.Add(BookingsTab.LoadAsync());
            if (TabUsers) loadTasks.Add(UsersTab.LoadAsync());
            if (TabMaintenance) loadTasks.Add(MaintenanceTab.LoadAsync());
            if (TabBookingServices) loadTasks.Add(BookingServicesTab.LoadAsync());

            await Task.WhenAll(loadTasks);

            Status = $"Успешно загружено таблиц: {loadTasks.Count}. Последнее обновление:
{DateTime.Now:HH:mm:ss}";
        }
        catch (Exception ex)
        {
            Status = $"Ошибка при загрузке: {ex.Message}";
            MessageBox.Show($"Ошибка: {ex.Message}", "Ошибка БД", MessageBoxButton.OK,
MessageBoxImage.Error);
        }
        finally
        {
            IsLoading = false;
        }
    }

    private void OpenDashboard()
    {
        if (!TabDashboard)
        {
            return;
        }

        var service = new DashboardService(_databaseManager);
        var viewModel = new DashboardViewModel(service);
        var window = new Views.DashboardWindow(viewModel)
        {
            Owner = Application.Current.MainWindow
        };
        window.Show();
    }

    private void OpenRoleHelp()
    {
        var window = new RoleHelpWindow(_role, CurrentUserRole)
        {
            Owner = Application.Current.MainWindow
        };
        window.Show();
    }

    private async Task CreateBackupAsync()
    {
        if (!CanEditAdminOnly)
        {
            return;
        }

        var dialog = new SaveFileDialog
        {
            Title = "Сохранить backup базы данных",
            Filter = "SQL backup (*.sql)|*.sql|Все файлы (*.*)|*.*",
            FileName =
$"{_databaseManager.Database}_backup_{DateTime.Now:yyyyMMdd_HHmmss}.sql",
            AddExtension = true,
            DefaultExt = ".sql",
            OverwritePrompt = true
        };

        if (dialog.ShowDialog() != true)
        {
            return;
        }
```

```
        IsLoading = true;
        Status = "Создание backup базы данных...";
        try
        {
            await _backupService.CreateBackupAsync(dialog.FileName);
            Status = $"Backup сохранен: {dialog.FileName}";
            MessageBox.Show("Backup успешно создан.", "Backup", MessageBoxButton.OK,
MessageBoxImage.Information);
        }
        catch (Exception ex)
        {
            Status = $"Ошибка backup: {ex.Message}";
            MessageBox.Show(ex.Message, "Ошибка backup", MessageBoxButton.OK,
MessageBoxImage.Error);
        }
        finally
        {
            IsLoading = false;
        }
    }

    private async Task RestoreBackupAsync()
    {
        if (!CanEditAdminOnly)
        {
            return;
        }

        var dialog = new OpenFileDialog
        {
            Title = "Выбрать backup базы данных",
            Filter = "SQL backup (*.sql)|*.sql|Все файлы (*.*)|*.*",
            CheckFileExists = true,
            Multiselect = false
        };

        if (dialog.ShowDialog() != true)
        {
            return;
        }

        var confirmation = MessageBox.Show(
            "Восстановление выполнит SQL из выбранного backup-файла. Текущие данные могут быть
заменены. Продолжить?",
            "Восстановление backup",
            MessageBoxButton.YesNo,
            MessageBoxImage.Warning);

        if (confirmation != MessageBoxResult.Yes)
        {
            return;
        }

        IsLoading = true;
        Status = "Восстановление базы данных из backup...";
        try
        {
            await _backupService.RestoreBackupAsync(dialog.FileName);
            Status = "Backup успешно восстановлен. Обновляю данные...";
            IsLoading = false;
            await LoadData();
            MessageBox.Show("Backup успешно восстановлен.", "Backup", MessageBoxButton.OK,
MessageBoxImage.Information);
        }
        catch (Exception ex)
        {
            Status = $"Ошибка восстановления backup: {ex.Message}";
```

```
            MessageBox.Show(ex.Message, "Ошибка восстановления backup", MessageBoxButton.OK,
MessageBoxImage.Error);
        }
        finally
        {
            IsLoading = false;
        }
    }

    private void SetStatus(string status)
    {
        Status = status;
    }

    private void ObserveCurrentCountCollections()
    {
        ClientsTab.Clients.CollectionChanged += OnTabCollectionChanged;
        RoomsTab.Rooms.CollectionChanged += OnTabCollectionChanged;
        EmployeesTab.Employees.CollectionChanged += OnTabCollectionChanged;
        ServicesTab.HotelServices.CollectionChanged += OnTabCollectionChanged;
        BookingsTab.Bookings.CollectionChanged += OnTabCollectionChanged;
        MaintenanceTab.Maintenances.CollectionChanged += OnTabCollectionChanged;
        BookingServicesTab.BookingLinks.CollectionChanged += OnTabCollectionChanged;
        UsersTab.Users.CollectionChanged += OnTabCollectionChanged;
    }
            private void OnTabCollectionChanged(object? sender,
NotifyCollectionChangedEventArgs e)
            {
                OnPropertyChanged(nameof(CurrentCount));
            }

            private void OnCurrentPageMetadataChanged()
            {
                OnPropertyChanged(nameof(CurrentPageTitle));
                OnPropertyChanged(nameof(CurrentPageIcon));
                OnPropertyChanged(nameof(CurrentCount));
            }

            private static string FormatRole(string role)
            {
                return role?.Trim().ToLowerInvariant() switch
                {
                    "admin" => "Администратор",
                    "manager" => "Менеджер",
                    "receptionist" => "Ресепшионист",
                    "housekeeper" => "Горничная",
                    "technician" => "Техник",
                    "operator" => "Оператор",
                    _ => string.IsNullOrWhiteSpace(role) ? "Пользователь" : role
                };
            }

            private static int GetDefaultTabIndex(string role)
            {
                return role?.Trim().ToLowerInvariant() switch
                {
                    "technician" => 5,
                    "housekeeper" => 6,
                    _ => 0
                };
            }
        }
    }
    using System;
    using System.Collections.ObjectModel;
    using System.Threading.Tasks;
    using System.Windows;
    using System.Windows.Input;
    using Hotel_Application.Helpers;
```

```
using Hotel_Application.Models;
using Hotel_Application.Services;

namespace Hotel_Application.ViewModels
{
    public class UsersViewModel : TabViewModelBase
    {
        private readonly UserService _userService;
        private User? _selectedUser;

        public ObservableCollection<User> Users { get; } = new
ObservableCollection<User>();

        public User? SelectedUser
        {
            get => _selectedUser;
            set { _selectedUser = value; OnPropertyChanged(); }
        }

        public ICommand AddUserCommand { get; }
        public ICommand EditUserCommand { get; }
        public ICommand DeleteUserCommand { get; }

        public UsersViewModel(UserService userService, Action<string> setStatus, Func<Task>
refreshAllAsync)
            : base(setStatus, refreshAllAsync)
        {
            _userService = userService;
            AddUserCommand = new RelayCommand(_ => AddUser());
            EditUserCommand = new RelayCommand(_ => EditUser());
            DeleteUserCommand = new AsyncRelayCommand(_ => DeleteUserAsync());
        }

        public override async Task LoadAsync()
        {
            var users = await _userService.GetUsersAsync();
            UpdateCollection(Users, users);
        }

        private void AddUser()
        {
            var viewModel = new AddUserViewModel(_userService);
            var window = new Views.AddUserWindow(viewModel)
            {
                Owner = Application.Current.MainWindow
            };
            viewModel.OnCompleted += async () => await RefreshAllAsync();
            window.ShowDialog();
        }

        private void EditUser()
        {
            if (SelectedUser == null)
            {
                MessageBox.Show("Выберите пользователя для редактирования.",
                    "Информация", MessageBoxButton.OK, MessageBoxImage.Information);
                return;
            }

            var viewModel = new EditUserViewModel(_userService, SelectedUser);
            var window = new Views.EditUserWindow(viewModel)
            {
                Owner = Application.Current.MainWindow
            };
            viewModel.OnCompleted += async () => await RefreshAllAsync();
            window.ShowDialog();
        }

        private async Task DeleteUserAsync()
```

```
        {
            if (SelectedUser == null)
            {
                MessageBox.Show("Выберите пользователя для удаления.",
                    "Информация", MessageBoxButton.OK, MessageBoxImage.Information);
                return;
            }

            var result = MessageBox.Show(
                $"Удалить пользователя «{SelectedUser.Username}»?",
                "Подтверждение",
                MessageBoxButton.YesNo,
                MessageBoxImage.Question);

            if (result != MessageBoxResult.Yes)
            {
                return;
            }

            long callerId = SessionService.CurrentUser!.Id;
            var (success, message) = await _userService.DeleteUserAsync(SelectedUser.Id,
callerId);

            if (!success)
            {
                MessageBox.Show(message, "Ошибка", MessageBoxButton.OK,
MessageBoxImage.Error);
                return;
            }

            await RefreshAllAsync();
        }
    }
}
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Hotel_Application.Helpers;
using Hotel_Application.Models;
using MySqlConnector;

namespace Hotel_Application.Services
{
    public class UserService
    {
        private readonly DatabaseManager _databaseManager;

        public UserService(DatabaseManager databaseManager)
        {
            _databaseManager = databaseManager;
        }

        public async Task<User?> AuthenticateAsync(string username, string password)
        {
            using var connection = _databaseManager.GetConnection();
            await connection.OpenAsync();

            User? user;
            using (var command = new MySqlCommand("sp_User_GetForAuthentication",
connection))
            {
                command.CommandType = System.Data.CommandType.StoredProcedure;
                command.Parameters.AddWithValue("@p_username", username);

                using var reader = await command.ExecuteReaderAsync();
                if (!await reader.ReadAsync())
                {
                    return null;
                }
```

```
                user = MapUser(reader, includePasswordHash: true);
            }

            if (!PasswordHasher.VerifyPassword(password, user.PasswordHash))
            {
                return null;
            }

            if (PasswordHasher.NeedsRehash(user.PasswordHash))
            {
                await UpdatePasswordHashAsync(connection, user.Id,
PasswordHasher.HashPassword(password));
            }

            await UpdateLastLoginAsync(connection, user.Id);
            user.LastLogin = DateTime.Now;
            user.PasswordHash = string.Empty;
            return user;
        }

        public async Task<(bool Success, string Message, User? User)> RegisterAsync(
            string username,
            string password,
            string role,
            int? idStaff = null)
        {
            using var connection = _databaseManager.GetConnection();
            await connection.OpenAsync();

            if (await UsernameExistsAsync(connection, username))
            {
                return (false, "Пользователь с таким логином уже существует.", null);
            }

            try
            {
                using var command = new MySqlCommand("sp_User_Create", connection);
                command.CommandType = System.Data.CommandType.StoredProcedure;
                command.Parameters.AddWithValue("@p_username", username);
                command.Parameters.AddWithValue("@p_password_hash",
PasswordHasher.HashPassword(password));
                command.Parameters.AddWithValue("@p_role", role);
                command.Parameters.AddWithValue("@p_id_staff",
                    idStaff.HasValue ? (object)idStaff.Value : DBNull.Value);

                var result = await command.ExecuteScalarAsync();
                var userId = Convert.ToInt64(result);

                return (true, "Пользователь успешно создан.", new User
                {
                    Id = userId,
                    Username = username,
                    Role = role,
                    LinkedStaffId = idStaff
                });
            }
            catch (MySqlException ex) when (ex.Number == 1062)
            {
                return (false, "Пользователь с таким логином уже существует.", null);
            }
            catch (MySqlException ex) when (ex.Number == 1644)
            {
                return (false, ex.Message, null);
            }
        }

        public async Task<List<User>> GetUsersAsync()
        {
```

```
            var results = new List<User>();

            using var connection = _databaseManager.GetConnection();
            await connection.OpenAsync();

            using var command = new MySqlCommand("sp_User_GetAll", connection);
            command.CommandType = System.Data.CommandType.StoredProcedure;
            using var reader = await command.ExecuteReaderAsync();

            while (await reader.ReadAsync())
            {
                results.Add(MapUser(reader, includePasswordHash: false));
            }

            return results;
        }

        private static async Task<bool> UsernameExistsAsync(MySqlConnection connection,
string username)
        {
            using var command = new MySqlCommand("sp_User_Exists", connection);
            command.CommandType = System.Data.CommandType.StoredProcedure;
            command.Parameters.AddWithValue("@p_username", username);

            var result = await command.ExecuteScalarAsync();
            return Convert.ToInt32(result) > 0;
        }

        private static async Task UpdateLastLoginAsync(MySqlConnection connection, long
userId)
        {
            using var command = new MySqlCommand("sp_User_UpdateLastLogin", connection);
            command.CommandType = System.Data.CommandType.StoredProcedure;
            command.Parameters.AddWithValue("@p_id_user", userId);
            await command.ExecuteNonQueryAsync();
        }

        private static async Task UpdatePasswordHashAsync(
            MySqlConnection connection,
            long userId,
            string passwordHash)
        {
            using var command = new MySqlCommand("sp_User_UpdatePasswordHash", connection);
            command.CommandType = System.Data.CommandType.StoredProcedure;
            command.Parameters.AddWithValue("@p_id_user", userId);
            command.Parameters.AddWithValue("@p_password_hash", passwordHash);
            await command.ExecuteNonQueryAsync();
        }

        public async Task<(bool Success, string Message)> DeleteUserAsync(long idUser, long
callerId)
        {
            using var connection = _databaseManager.GetConnection();
            await connection.OpenAsync();

            try
            {
                using var command = new MySqlCommand("sp_User_Delete", connection);
                command.CommandType = System.Data.CommandType.StoredProcedure;
                command.Parameters.AddWithValue("@p_id_user", idUser);
                command.Parameters.AddWithValue("@p_caller_id", callerId);

                using var reader = await command.ExecuteReaderAsync();
                if (await reader.ReadAsync())
                {
                    return (true, reader.GetString("Message"));
                }

                return (false, "Неизвестная ошибка при удалении.");
```

```
            }
            catch (MySqlException ex)
            {
                return (false, ex.Message);
            }
        }

        public async Task<(bool Success, string Message)> UpdateUserRoleAsync(
            long idUser,
            string newRole,
            int? idStaff)
        {
            using var connection = _databaseManager.GetConnection();
            await connection.OpenAsync();

            try
            {
                using var command = new MySqlCommand("sp_User_UpdateRole", connection);
                command.CommandType = System.Data.CommandType.StoredProcedure;
                command.Parameters.AddWithValue("@p_id_user", idUser);
                command.Parameters.AddWithValue("@p_new_role", newRole);
                command.Parameters.AddWithValue("@p_id_staff",
                    idStaff.HasValue ? (object)idStaff.Value : DBNull.Value);

                using var reader = await command.ExecuteReaderAsync();
                if (await reader.ReadAsync())
                {
                    return (true, reader.GetString("Message"));
                }

                return (false, "Неизвестная ошибка.");
            }
            catch (MySqlException ex)
            {
                return (false, ex.Message);
            }
        }

        public async Task<List<StaffListItem>> GetActiveStaffAsync()
        {
            var results = new List<StaffListItem>();
            using var connection = _databaseManager.GetConnection();
            await connection.OpenAsync();

            using var command = new MySqlCommand("sp_Staff_GetAllActive", connection);
            command.CommandType = System.Data.CommandType.StoredProcedure;
            using var reader = await command.ExecuteReaderAsync();

            while (await reader.ReadAsync())
            {
                results.Add(new StaffListItem
                {
                    Id = reader.GetInt32("id_Staff"),
                    FullName = $"{reader.GetString("last_name_staff")}
{reader.GetString("first_name_staff")}",
                    JobTitle = reader.GetString("job_title"),
                    HasUser = reader.GetInt32("has_user") == 1
                });
            }

            return results;
        }

        private static User MapUser(MySqlDataReader reader, bool includePasswordHash)
        {
            var staffOrdinal = SafeGetOrdinal(reader, "id_staff");
            var staffNameOrdinal = SafeGetOrdinal(reader, "staff_full_name");
            var user = new User
            {
```

```
                Id = reader.GetInt64("id_user"),
                Username = reader.GetString("username"),
                Role = reader.GetString("role"),
                LastLogin = reader.IsDBNull(reader.GetOrdinal("last_login"))
                    ? null
                    : reader.GetDateTime("last_login"),
                LinkedStaffId = staffOrdinal >= 0 && !reader.IsDBNull(staffOrdinal)
                    ? reader.GetInt32(staffOrdinal)
                    : null,
                LinkedStaffName = staffNameOrdinal >= 0 &&
!reader.IsDBNull(staffNameOrdinal)
                    ? reader.GetString(staffNameOrdinal)
                    : string.Empty
            };

            if (includePasswordHash)
            {
                user.PasswordHash = reader.GetString("password_hash");
            }

            return user;
        }

        private static int SafeGetOrdinal(MySqlDataReader reader, string fieldName)
        {
            try
            {
                return reader.GetOrdinal(fieldName);
            }
            catch (IndexOutOfRangeException)
            {
                return -1;
            }
        }
    }
}
```